



From pixels to rating—A semiautomated system linking multi-damage segmentation and condition rating in concrete bridge inspections

Rona Firdes Çelik¹ | Vedhus Hoskere² | Sylvia Kessler¹

¹Engineering Materials and Building Preservation,
Helmut-Schmidt-University/University of
the Federal Armed Forces Hamburg,
Hamburg, Germany

²Department of Electrical and Computer
Engineering, University of Houston,
Houston, Texas, USA

Correspondence

Rona Firdes Çelik, Chair of Engineering
Materials and Building Preservation,
Helmut-Schmidt-University/University of
the Federal Armed Forces Hamburg,
Holstenhofweg 85, 22043 Hamburg,
Germany.
Email: celikf@hsu-hh.de

Abstract

While machine learning (ML) has advanced image-based damage detection, a critical gap remains: the automated translation of detected damage into standardized condition ratings used in structural assessments. Most existing approaches stop at semantic segmentation, overlooking the damage rating step essential for practical inspections. This paper presents a semiautomated system that bridges this gap by linking multi-label damage segmentation with condition rating prediction. Our contributions are: (1) a data-driven label taxonomy for damage segmentation, derived from statistical and semantic analysis of 2.2 million inspection records, and designed to support downstream condition rating; (2) a pipeline for converting textual inspection records into structured training data for automated condition rating, and a set of custom bidirectional long short-term memory (LSTM) models achieving up to 99 % F1-score on this task; and (3) a reference system architecture integrating image segmentation and text-based damage rating within an interactive 3D inspection interface. The system demonstrates how integrating damage detection and condition rating within an interactive 3D interface can streamline inspection documentation and enhance decision support for concrete structures. Developed in compliance with German inspection standards and designed for adaptability, the system architecture offers a transferable framework for embedding ML-based automation into digital inspection workflows, ensuring that all components, from damage detection to condition rating, are aligned in an end-to-end process.

1 | INTRODUCTION

The structural condition of infrastructure assets is routinely assessed through visual inspections aimed at identifying potential damage that could compromise functionality or safety. Typical forms of damage include cracks,

honeycombing, spalling, or other unintended material inhomogeneities. To date, these inspections are predominantly carried out by trained bridge inspectors. However, the identification and quantification of such damage remain highly susceptible to human subjectivity and variability (Graybeal et al., 2002). Following the detection of

This is an open access article under the terms of the [Creative Commons Attribution](https://creativecommons.org/licenses/by/4.0/) License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited.

© 2025 The Author(s). *Computer-Aided Civil and Infrastructure Engineering* published by Wiley Periodicals LLC on behalf of Editor.

damage, its accurate localization and quantification are critical steps in assigning the affected component or structure to a discrete condition class. This classification forms the basis for maintenance prioritization and long-term asset management strategies.

Inspection results are typically recorded and managed within systems such as Bridge Management Systems (BMS), which function as valuable repositories of structural condition data. These databases form an essential foundation for the integration of digital tools and decision-making processes (Marsili et al., 2025). In recent years, the entire workflow of data collection, documentation, and storage has been undergoing a gradual digital transformation.

Given the vast scale of infrastructure networks and the large number of inspections conducted annually, the digitalization of the end-to-end process, from automated damage detection to quantification and condition rating, holds significant potential to improve efficiency, consistency, and objectivity. The integration of digital tools, such as image-based damage detection using machine learning (ML), promises to support a more scalable and standardized approach to condition assessment.

The ML application in bridge inspections aims at automating inspection processes to provide bridge managers with reliable and structured assessments of structural conditions. Visual damage detection, including classification, bounding-box detection, and segmentation of various damage types, is the fundamental step in automated inspection workflows. However, moving beyond detection to standardized damage evaluation (condition rating) remains a significant challenge.

Current research in ML for bridge inspections predominantly focuses on crack detection, driven largely by the complexity involved in recognizing fine objects. Despite their importance, cracks represent only a subset of structural damage encountered during inspections. Efficient automated systems require recognizing multiple, often overlapping types of structural damage, yet research addressing multi-damage semantic segmentation in depth is comparatively scarce. On the one hand, there is limited discourse on how to systematically document detected damage types, given their inherent complexity, hierarchical nature (severity), and frequent co-occurrence. On the other hand, this research field has been constrained by the lack of a large, diverse, and well-annotated image data set of damage in reinforced concrete structures.

Defining effective damage classes¹ is challenging. Damages exhibit significant variability, often involving mul-

tle overlapping and cascading categories. For instance, the damage type “exposed reinforcement bar” involves corrosion-induced concrete spalling, creating ambiguity in labeling decisions. Such scenarios demand carefully designed damage labeling strategies that balance comprehensiveness and practical feasibility. Two critical factors guide optimal damage labeling: (1) efficient class definitions that cover relevant damage types without leading to excessive data fragmentation, and (2) compatibility with standardized damage labeling schemes used in official BMSs, such as Pontis/BrM in the United States or SIB-Bauwerke in Germany to name two.

Importantly, many BMS databases not only standardize damage labels but also store extensive structured condition rating data associated with each recorded damage instance. Despite the availability of these data, no robust, integrated workflow currently exists to connect automated damage detection directly with structured BMS-derived condition rating. While existing literature addresses concepts around digitalizing bridge inspections using UAV-based or mobile application-assisted methods to localize damages within 3D models, there are no studies integrating comprehensive damage rating based on standardized damage definitions, locations, and dimensions.

To address this critical research gap, we focus on bridging the divide between image-based multi-damage segmentation and structured, text-based condition rating. We propose a semiautomated inspection framework that integrates a damage segmentation model with natural language processing (NLP)-based rating models trained on German bridge inspection data. The system operates within an interactive 3D bridge model interface, enabling inspectors to localize damage, upload images, and receive real-time ML-based segmentation and rating results, as illustrated in Figure 1. Human oversight is maintained through manual entry of additional damage information, such as quantity and description, and the ability to review and correct ML outputs, ensuring both accuracy and practical usability.

The question guiding our work is: *How can automated multi-damage segmentation be effectively integrated with condition rating models to support structured and reliable damage documentation in bridge inspections?*

Our main contributions are as follows:

- (1) **Data-driven label taxonomy for bridge damage segmentation:** We present a structured and scalable taxonomy of bridge damage classes, grounded in statistical and semantic analysis of over 2.2 million records from the German BMS (SIB-Bauwerke). Designed to align with BMS labels, the taxonomy supports an efficient annotation strategy and covers 22 standardized damage types in the BMS. Building on this, we

¹ In this paper, we use the terms *damage class* and *damage type*. *Damage class* refers to categories in ML-based classification and *damage type* refers to categories defined in BMSs.

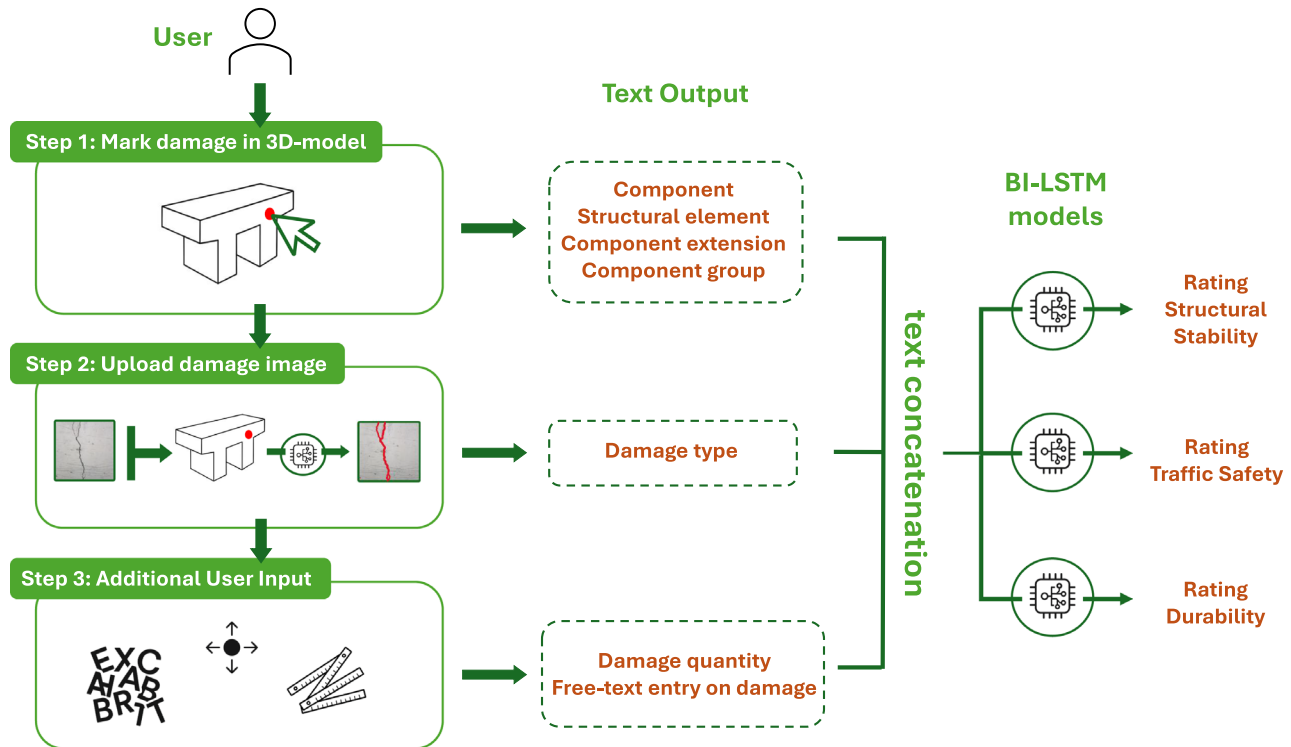


FIGURE 1 Workflow diagram of the proposed framework, illustrating the end-to-end processing pipeline from bridge damage input to condition rating.

developed a custom data set of 2070 annotated images across 8 core damage classes, enabling standardized model training and ensuring compatibility with the downstream condition rating task

- (2) **Automated Damage Condition Rating:** We introduce a preprocessing pipeline that converts raw inspection records into a structured format suitable for learning condition ratings. This includes domain-specific standardization of terminology and label alignment for both damage types and quantities. To address severe class imbalance, we combine bi-directional LSTM models with targeted sampling and loss-based training strategies. The resulting models demonstrate strong predictive performance and provide breakdowns of typical misclassifications, offering actionable insights to support inspector decision making.
- (3) **System Architecture for Integrated 3D Inspection:** We present a semiautomated system that integrates image-based damage segmentation and text-based condition rating within an interactive 3D bridge model. Inspectors localize damage directly in the model, triggering automatic retrieval of structural metadata. Images are segmented, damage types extracted, and enriched with additional inputs before being processed by NLP models to predict condition ratings. The architecture supports human–AI

collaboration: Inspectors can review and correct ML outputs at each stage, ensuring that even incorrect segmentation do not affect downstream assessment.

The rest of this paper is structured as follows: Section 2 reviews related work on bridge inspection automation, multi-damage segmentation, and condition assessment. Section 3 outlines the overall system architecture. Section 4 presents our segmentation methodology, covering label taxonomy design, data set analysis, and model training on imbalanced data. Section 5 details the NLP-based condition rating approach, including data processing and modeling. Section 6 introduces the integrated inspection system and explains how segmentation results are post-processed and linked to condition rating models. Section 7 discusses the system's strengths, limitations, and future directions. Finally, Section 8 summarizes our findings and outlines implications for ML-based bridge inspection.

2 | RELATED WORK

2.1 | Digitalization of bridge inspection workflows

The digitalization of bridge inspection began with the shift from paper-based reporting to structured digital data man-

agement. Over time, this transition created a demand for more advanced field software. Today's inspection tools offer robust mobile applications that support offline data entry, provide built-in compliance guidance, and enable integrated image annotation.

Brighenti et al. (2024) provide an overview of recent practices in bridge management software and identify a fundamental limitation that persists: Data generation for condition rating still relies on manual inspection and subjective judgment. Current software organizes and manages input but does not automate the core tasks (Abdallah et al., 2022). Researchers have sought to extend these functionalities through mobile applications (Kruachotikul et al., 2021) and mixed reality devices, such as the HoloLens (Çelik et al., 2022; Karaaslan et al., 2019), which enable structured damage documentation supported by AI-assisted damage detection. In a related effort, multiple studies were conducted on integration of inspection and damage information into digital bridge models, particularly through frameworks based on Industry Foundation Classes (IFC) (Artus et al., 2022; Hühthwohl et al., 2018; Li et al., 2023; Sacks et al., 2018). These studies focus on developing bridge information schemas and workflows that integrate detected damage to a building information model (BIM), thereby enhancing interoperable and automated bridge inspection and management. Concurrently, NLP methods have been applied to extract structured data from legacy reports (Li et al., 2021; Liu and El-Gohary, 2017).

2.2 | Automated damage detection

To automate damage detection in images, researchers have turned to computer vision, and in particular semantic segmentation, which assigns a class label (e.g., “crack,” “spalling”) to every image pixel. This enables the extraction of quantitative information such as damage extent and location (Çelik et al., 2022; Kim & Cho, 2020; Liu et al., 2024; Yu et al., 2025). For efficient and safer data acquisition, especially in hard-to-access areas, aerial imagery captured by UAVs is commonly used in practice (La et al., 2015; Rakoczy et al., 2024; Spencer Jr et al., 2019).

Early segmentation work relied on encoder–decoder networks such as U-Net (Ronneberger et al., 2015), which remain a paradigm for designing effective segmentation architectures (Chen & He, 2022; Liu et al., 2025; Zhu et al., 2024). More recent architectures such as transformer-based models (Chung et al., 2023; Qin et al., 2024; Wang et al., 2025; Zhou et al., 2023) and large foundation models such as Segment Anything Model (SAM) (Azimi & Yang, 2024; Kirillov et al., 2023) have further improved accuracy.

Simultaneously detecting multiple damage types in a single scene is a challenging yet highly valuable capa-

bility for holistic bridge evaluation. Modern approaches also built multi-task learning architectures that train one network to recognize several damage categories, such as cracks, spalling, and corrosion, and even distinguish material classes in parallel (Hoskere, 2020; Zaker Esteghamati, 2024). By sharing feature representations across these related tasks, the models become both more robust and efficient. To support this research, specialized benchmarks like CODEBRIM (Mundt et al., 2019), dacl10k (Flotzinger et al., 2024), and the structural defect data set S2DS (Benz & Rodehorst, 2022) provide annotated, multi-damage data sets for training and validating multi-damage detection and segmentation methods.

Beyond image-based segmentation, ML has also been applied in related domains to detect nonvisual forms of damage in infrastructure. This includes applications in structural health monitoring (SHM) (Perez-Ramirez et al., 2019; Rafiei & Adeli, 2017, 2018; Javadinasab Hormozabad et al., 2021; Luleci et al., 2022) using sensors (Giglioni et al., 2023; Sajedi & Liang, 2022), as well as in non-destructive testing and evaluation (NDT, NDE), such as the analysis of ground-penetrating radar (GPR) data (Rosso et al., 2024), highlighting the potential for automation across diverse sensor modalities in SHM.

2.3 | Bridge digital twins (DTs) as a contextual framework

Informed by cyberphysical interfaces that convert data to a virtual replica, DTs can serve as a powerful tool for comprehensive assessment and documentation of bridge condition (Hoskere et al., 2025). DTs can be built on as-is geometry (LiDAR or photogrammetry) and semantically rich BIM (Ye et al., 2019). Recent research has automated DT creation through “Scan-to-Twin” processes that segment point clouds and generate parametric BIMs (Lu & Brilakis, 2019; Rahman & Hoskere, 2025; Xia et al., 2022). Once produced, these 3D representations can provide the necessary context to understand the damage from 2D segmentation. An open challenge remains identifying approaches that integrate this information in a DT to automate condition ratings.

2.4 | Data-driven and automated condition rating

A common goal of inspection is to inform condition ratings. Early efforts employed Markov chain models to forecast component state transitions based on historical inspection records (Cesare et al., 1992). Ranjith et al. (2013) extended these stochastic methods to timber bridge



elements, demonstrating applicability across material types. With the advent of large National Bridge Inventory (NBI) data sets, ensemble ML approaches have become dominant. Assaad and El-Adaway (2020) demonstrated that boosting algorithms, such as XGBoost, achieve superior accuracy in forecasting deck deterioration under varied traffic and environmental conditions. Building on this, Rashidi Nasab and Elzarka (2023) optimized XGBoost hyperparameters in a case study of Ohio bridges, yielding notable gains over baseline models. Recently, Landi et al. (2025) applied an unsupervised learning approach to cluster bridges into three degradation groups which enabled the development of a semi-Markov model for each group. These models capture the evolution of bridge conditions and were tested with NBI data. Kong et al. (2022) reported over 85% accuracy in multicomponent deterioration prediction using Random Forest on NBI-derived features. Early neural-network-based efforts date back to Huang (2010), who employed feed-forward ANNs to model deck condition transitions with promising results.

Despite these advances, key challenges remain. Most data-driven models still predict legacy, subjective ratings rather than the quantitative, element-level condition states defined in modern manuals. Moreover, truly unified, multicomponent rating systems at network scale capable of incorporating high-resolution imagery and rule-based thresholds remain an open frontier.

3 | SYSTEM ARCHITECTURE

This work presents a semiautomated inspection system that integrates image-based multi-damage detection with text-based condition rating. The system is built around a 3D bridge model viewer interface, which serves as the central platform linking these two functionalities into a seamless workflow from damage localization and segmentation to automated rating prediction.

The system is semiautomated, as it requires some user input from bridge inspectors, such as manual localization of the damage and entry of quantitative or descriptive metadata (e.g., damage dimensions). The rest of the damage analysis pipeline is handled automatically with embedded ML modules.

The core components of the application are as follows:

- **3D Viewer Interface:** The interface displays a predefined bridge model, which was manually designed using the Python library PyVista. Within this model, users can manually annotate damage locations by right-clicking directly on structural components. Each 3D model element stores metadata regarding its classification within the bridge management hierarchy, specifically, its com-

ponent, structural element, component extension, and component group.

- **Damage Annotation and Image Upload:** Upon damage location annotation, a damage information panel is triggered on the right side of the viewer. This panel allows users to upload an image corresponding to the annotated location. The image is then displayed in a preview window. Also, the following fields become available for input:
 1. **Damage type:** Automatically filled by the correct damage label after segmentation of uploaded image, but editable by the user if needed.
 2. **Text:** Free-text field to add further damage types, measurements (e.g., crack width), or qualitative observations.
 3. **Damage Quantity:** Drop-down menu with predefined categories of damage extent (see Section 5.1).
 4. **Component Metadata:** Fields for component, structural element, component extension, and component group are auto-filled based on the annotation's position in the 3D bridge model.
 5. **Condition Rating:** Includes a "Rating" button and empty fields labeled "S," "T," and "D," which represent the German condition rating criteria: structural stability (S), traffic safety (T), and durability (D).
- **Damage Segmentation:** Upon image upload, the segmentation model is executed in the background. The resulting segmentation masks are used to visualize detected damage and, most importantly, to identify the main damage type, which is automatically inserted into the respective field (e.g., *Damage type: Spalling with exposed rebar* in Figure 2). If the predicted damage type is incorrect or incomplete, inspectors can manually revise it, ensuring accuracy and maintaining full control over the assessment process.
- **Condition Rating Prediction:** When the "Rating" button is clicked on, all entered and ML-derived text data are aggregated into a structured input string. This string is input into three NLP models, each responsible for one of the three rating criteria, and the resulting ratings are automatically displayed in the corresponding fields.

Figure 2 provides a screenshot of the 3D viewer interface, showcasing the annotated bridge model and the associated damage information panel, which demonstrates the practical implementation of the workflow illustrated in Figure 1.

The complete processing pipeline, from annotation to automated condition rating, follows this sequence: manual damage location annotation → auto-filled location metadata (component, structural element, component extension, and component group) → damage image upload → damage segmentation on image → structured damage

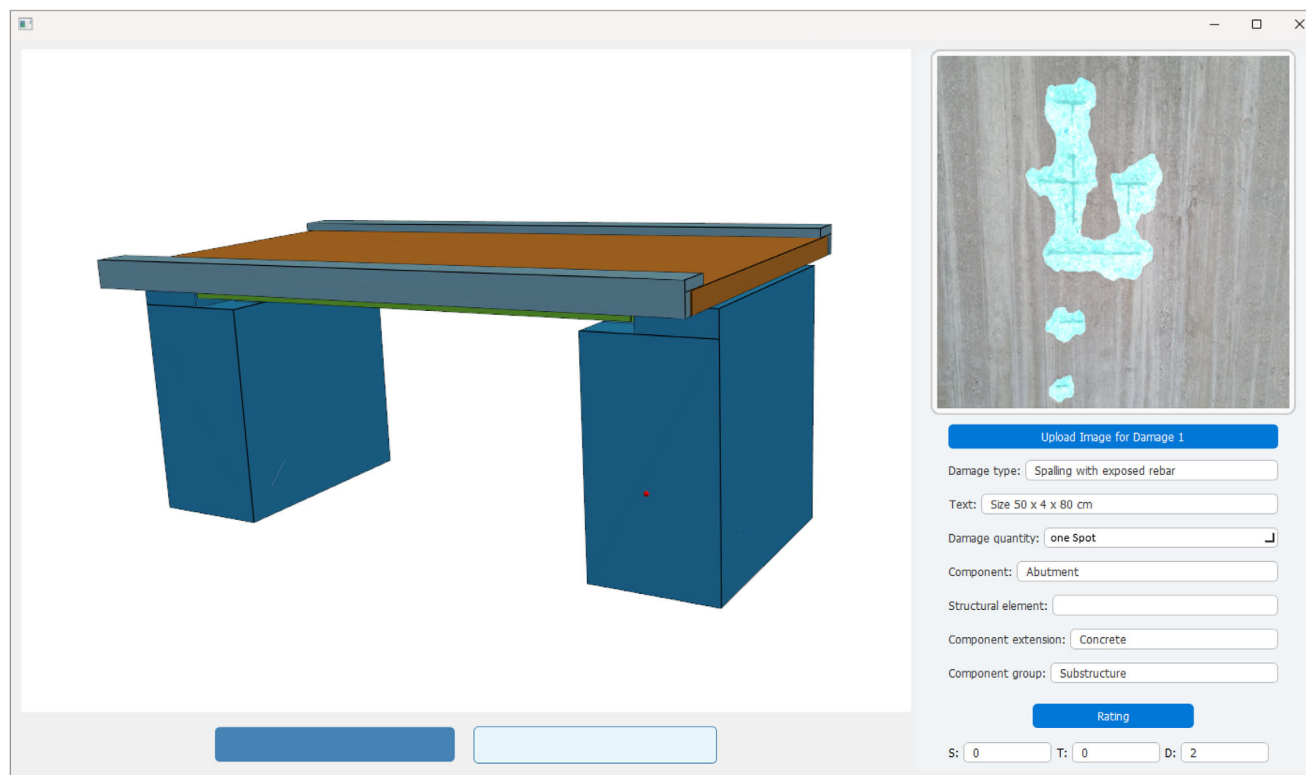


FIGURE 2 Screenshot of the 3D bridge model viewer interface with damage location annotation (red sphere on the right abutment) and damage information panel to the right. The original German descriptions in the panel were translated into English.

information text generation from entries in the information panel → condition rating prediction. A detailed description of the underlying processing logic is provided in Section 6, after the damage segmentation (Section 4) and condition rating modules (Section 5) are presented individually.

4 | MULTI-DAMAGE SEGMENTATION

Automated bridge inspection begins with visual damage recognition, forming the foundation for subsequent condition assessment. However, bridge damages are visually complex phenomena characterized by diverse shapes, textures, and scales. Recent studies on damage data sets have highlighted several recurring challenges in multi-damage segmentation. These include pronounced interclass similarity and intraclass variability; strong class imbalance, both at the pixel and image level; inconsistent annotations arising from vague or overlapping defect boundaries and differing annotator interpretations; and limited accessibility to concrete structures for data collection (Benz & Rodehorst, 2022; Çelik et al., 2022; Mundt et al., 2019; Flotzinger et al., 2024).

Moreover, many damages are compositional, that is, a single damage instance may involve multiple co-occurring

damage types. This complexity raises fundamental questions about the definition and labeling of damage classes for ML systems. For instance, the damage type exposed reinforcement bar may be labeled in multiple ways: (1) as a single compound class (spalling with exposed reinforcement), (2) as two independent classes (spalling and reinforcement bar), or (3) as spalling and corrosion. While the latter two approaches enable more modular labeling, they require additional post-processing steps to infer composite damages. Together, these factors complicate the development of generalizable models and emphasize the need for carefully designed taxonomies and quality-controlled annotation workflows.

To address these challenges, we define two key principles in defining damage classes. First, the taxonomy must strike a balance between coverage and compactness. The label set should cover the majority of observed damages while minimizing the number of classes to avoid data fragmentation and class overlap. Second, the label set must be compatible with labels existing in BMSs. Systems such as SIB-Bauwerke (Germany) or Pontis/BrM (USA) store large-scale annotated inspection data and define standardized damage labels. Aligning our label definitions with standardized damage labels in BMSs facilitates downstream integration of developed approaches.



The following subsections describe our approach to damage label definition, annotation methodology, training data preparation, and model architecture for multi-damage segmentation.

4.1 | Damage label definition

ML-based damage recognition requires careful label design, as it directly affects both the annotation process and the performance of ML models. Although a fully automated inspection ideally includes recognition of all damage types, this is currently impractical due to data availability and model complexity. In this study, we define a set of eight representative damage classes based on both statistical and semantic analysis of bridge inspection records.

We based this selection on a CSV text data extracted from the German BMS SIB-Bauwerke. These data were provided by the road construction authority of the German federal state North Rhine-Westphalia. It is tabular and contains 2,237,628 entries, each documenting a single instance of structural damage. The damage types are saved in a dedicated column. A statistical analysis has resulted in 297 distinct damage types, each with a relatively small share of data samples. The most common damage type *missing* accounts for 8.24 % of samples. This damage type indicates all kinds of objects to structures that are missing, like a missing safety nut on the railing, a missing paving stone on the embankment, or the missing bitumen seal between the shoulder and the road surface. The second most common damage type is *spalling* (5.38 %), closely followed by *cracks with 0.1 mm ≤ 0.2 mm width* (5.33 %). Other damage types with a data share larger than 1 % are presented in descending order: *spalling with exposed rebar* (4.89 %), *slightly rusted* (3.74 %), *exposed rebar* (3.62 %), *cracked* (e.g., joint tape or asphalt) (3.31 %), *defective* (this is a general term, that may sum up multiple damage to an object like cracks and cavities in one spot, or refer to defective structure equipment like bird protection) (2.87 %), *cavity* (2.35 %), *vegetated* (2.19 %), *open* (e.g., expansion joint) (2.06 %), *crack with 0.2 mm ≤ 0.4 mm width* (2.05 %), *efflorescence* (1.57 %), *dirt accumulation* (1.48 %), *loose* (e.g., screws, lighting, anchor plates) (1.37 %), *cracks with < 0.1 mm width* (1.27 %), *not in accordance with professional standards* (1.22 %), *damaged due to external impact* (1.21 %), *not adhering* (e.g., joint sealant) (1.02 %), *impact damage* (1.01 %), and *does not comply with the applicable regulations* (1.01 %).

Many of these labels are either ambiguous (e.g., *missing*, *open*, *defective*) or overly specific (e.g., different crack width thresholds), posing challenges for image-based annotation. Therefore, we applied a two-step process to

define a label set that balances semantic clarity and statistical feasibility. First, we selected the most frequent and visually distinct damage types: *crack*, *spalling*, *spalling with exposed rebar*, *slightly rusted*, *exposed rebar*, *vegetated*, and *efflorescence*. All crack types, regardless of width, were merged into a single class *crack* to reduce fragmentation. Second, we extended the label set by reviewing less frequent but semantically important damage types (below 1 % frequency), prioritizing those that (a) are feasible to collect in sufficient quantity, and (b) commonly appear in combination with other damages, for example, *crack with moisture ingress and rust color*. This resulted in the inclusion of *moisture ingress*, *moss*, *rust flag*, and *honeycomb*.

To reduce data fragmentation by high number of labels, we introduced two criteria for efficient label definition:

Visual Intersections and Semantic Grouping

Several damage types share common visual characteristics. For example, *spalling with exposed rebar*, *slightly rusted*, and *exposed rebar* all involve corrosion. These were merged under a general class *corrosion*. While this results in the loss of fine-grained labels, such as differentiating *exposed rebar* from general rusting, these distinctions can be recovered through rule-based postprocessing. For instance, if a *corrosion* mask spatially overlaps with a *spalling* mask, we infer the presence of *exposed rebar*. But also in this case, the damage type *slightly rusted* can only be derived when the corresponding metal objects are also segmented. For now, we introduce only the damage class *corrosion*. More nuanced corrosion distinctions are reserved for future work.

Data Collection Feasibility Labels such as *vegetated* were excluded despite their relevance, as data collection seemed disproportionately difficult relative to other classes.

Based on these principles, we defined eight primary damage classes: *spalling*, *crack*, *corrosion*, *honeycomb*, *rust flag*, *moss*, *moisture ingress*, and *efflorescence*. In addition, compound damage types defined in SIB-Bauwerke can be inferred through postprocessing by analyzing mask intersections. This allows the derivation of five additional damage classes:

- (1) *spalling with exposed rebar*
- (2) *honeycomb with exposed rebar*
- (3) *moisture ingress with efflorescence*
- (4) *crack with moisture ingress*
- (5) *crack with moisture ingress and rust color*

Furthermore, the general class *crack* subsumes nine specific damage types defined in SIB-Bauwerke, including:

- (1) *crack with $0.1 \text{ mm} \leq 0.2 \text{ mm}$ width*
- (2) *crack with $0.2 \text{ mm} \leq 0.4 \text{ mm}$ width*
- (3) *crack with $0.4 \text{ mm} \leq 1 \text{ mm}$ width*
- (4) *crack with $\geq 1 \text{ mm}$ width*
- (5) *reopened repaired crack*
- (6) *cracks*
- (7) *torn*
- (8) *cracked*
- (9) *cracked and hollow sounding*

The documentation and use of these damage types in condition rating are discussed in Section 5.1.

4.2 | Damage data annotation approach

In the previous section on damage label definition, we implicitly assumed that at least bounding-box detection or object segmentation is available, as some damage types are derived through postprocessing, that is, by checking intersections between predicted masks. Certain damage types, such as *cracks*, require object segmentation, as geometric attributes like crack width or extent can be extracted in downstream tasks. Other damage types, such as *efflorescence*, may only require detection or even classification, since their documentation depends primarily on presence and location, not geometric measurements.

Object segmentation requires substantially more annotation effort than bounding-box or classification-based labeling. It is also more error-prone, especially for complex damage shapes, as annotator judgments can vary significantly when delineating the exact contours of damage instances. While tools like SAM2 (Ravi et al., 2024) offer faster labeling, their effectiveness is limited for damage data, which differs significantly from the many everyday objects these models were trained to segment.

Nonetheless, when compound damage types are to be derived by postprocessing segmentation masks, object segmentation offers the most appropriate foundation, particularly for irregularly shaped objects like cracks or moisture ingress, because it enables pixel-wise analysis of mask intersections. For these reasons, we selected segmentation as the annotation strategy for all damage classes to ensure reliable postprocessing.

Annotation was carried out by using SAM2, which proved most effective for *spalling*. For *crack* damage, satisfactory results were obtained only when small regions were manually marked. Overall, SAM2 worked best in cases where the image background was smooth and free of noise. However, many damage instances, such as mois-

ture ingress, lack sharp visual boundaries and blend into adjacent materials or background textures, limiting the usefulness of SAM2 across all damage types. Thus, despite moderate gains in labeling efficiency, manual annotation remained necessary for most classes.

4.3 | Training data

The data set prepared for this study has 2070 images of which 1535 images were collected from various concrete structures in Germany, Austria, and Belgium. 246 damage images of various concrete structures were provided by the German engineering company Kempen Krause Ingenieure GmbH. Additional crack images, that is, 289 images were added from the Bochum Crack Data set (BCDS) (Çelik & König, 2022). Except for the BCDS images which are of size 512×512 px, the images were first resized to 1024×1024 px for annotation and after annotation to 512×512 px. The initial size of 1024×1024 px was chosen in order to not lose information on fine and small damage when annotating.

Figure 3 gives a visual overview of the defined damage classes and Figure 4 visualizes the compound damage types as presented in Section 4.1. To better understand the data set and the training results, we provide an overview of the data distribution in Figures 5 to 7.

Figure 5 illustrates the distribution of damage masks across different classes within the data set. The majority of damage masks belong to the crack class, comprising 38.3 % of the total. Following at a significant distance are masks representing efflorescence (20.4 %), corrosion (15.1 %), and moisture ingress (11.8 %). Less commonly, the data set contains rust flag masks (5.2 %), spalling masks (4.7 %), and moss masks (4.3 %). The least represented damage class is honeycomb, accounting for only 0.2 % of the masks. This distribution clearly indicates a significant data imbalance within the data set, with a notable difference of 38.1 % between the most frequently occurring damage class (crack) and the least occurring damage class (honeycomb).

Figure 6 illustrates the distribution of damage classes across the images, highlighting the 10 most frequent damage class compositions within the data set. Notably, 16.04 % of the images contain only the damage class crack, which can be attributed to the crack images from the BCDS data set. Other single damage classes appearing among the top 10 are corrosion (7.0 %), efflorescence (5.51 %), moisture ingress (5.51 %), and rust flag (1.64 %). The most frequent compound damage composition consists of moisture ingress and efflorescence, accounting for 7.83 % of the images. This is followed by the combination of crack, moisture ingress, and efflorescence (4.11 %), crack and



FIGURE 3 Example images representing the defined damage classes. The top row (from left to right) includes spalling, crack, corrosion, and honeycomb. The bottom row (from left to right) includes rust flag, moss, moisture ingress, and efflorescence.



FIGURE 4 Example images illustrating compound damage types. From left to right: spalling with exposed rebar, honeycomb with exposed rebar, moisture ingress with efflorescence, crack with moisture ingress, and crack with moisture ingress and rust discoloration.

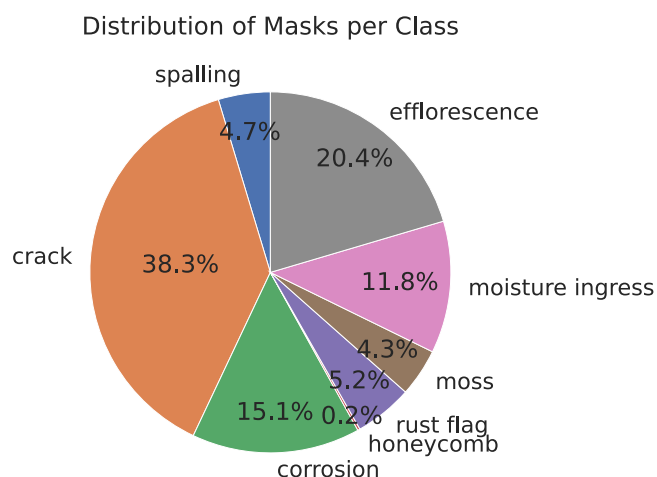


FIGURE 5 Pie chart illustrating the distribution of damage masks across different damage classes within the data set.

efflorescence (2.17%), crack, rust flag, moisture ingress, and efflorescence (1.79%), and finally, spalling, crack, moisture ingress, and efflorescence (1.74%).

Figure 7 presents boxplots illustrating the relative size distribution of damage masks for each damage class. In general, the majority of the damage masks occupy less than 1.0% of the image area, though there is substantial variability, as there are numerous outliers. Among the classes, honeycomb masks stand out as being larger, with the 25th, 50th (median), and 75th percentiles at approximately 0.13%, 0.31%, and 0.68%, respectively. Larger damage objects are better for accurate detection. This is advantageous for honeycomb detection given that honeycomb is the least represented damage class in the data set (Figure 5), compensating slightly for its low frequency. Conversely, cracks have the smallest image share, typically below 0.015% with a narrow interquartile range. However,

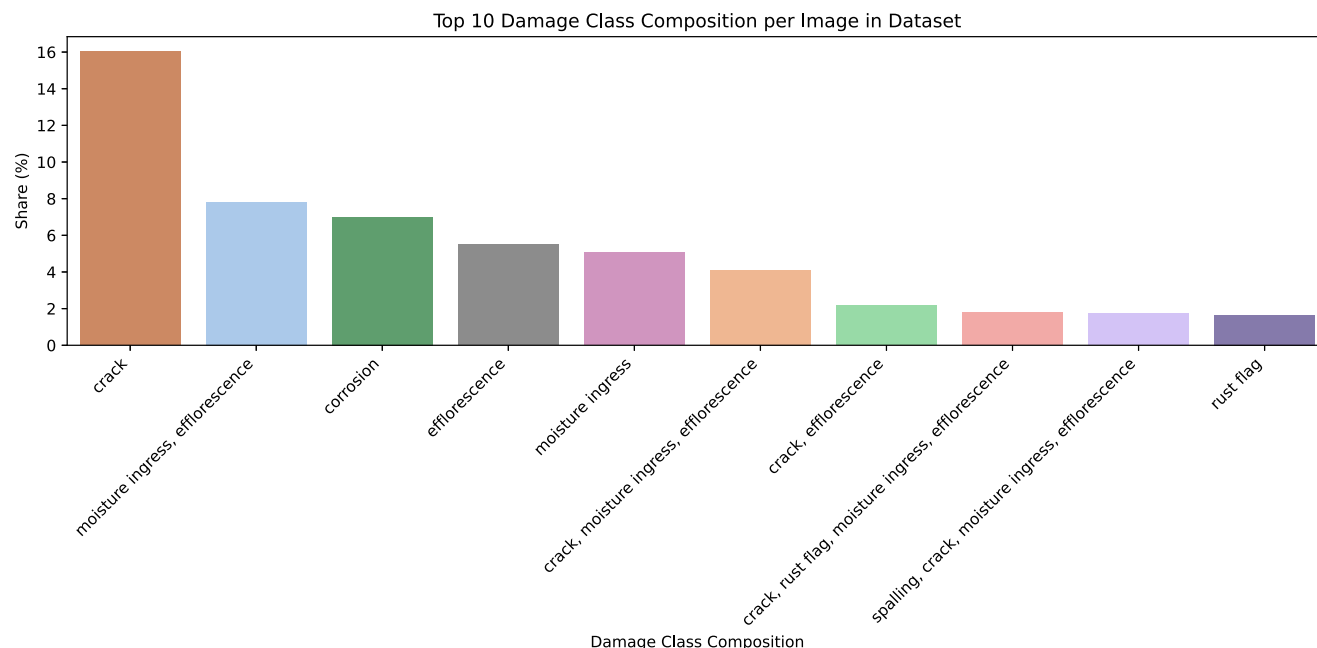


FIGURE 6 Bar chart showing the top 10 most frequent damage class compositions in the data set, presented as a percentage of the total image count.

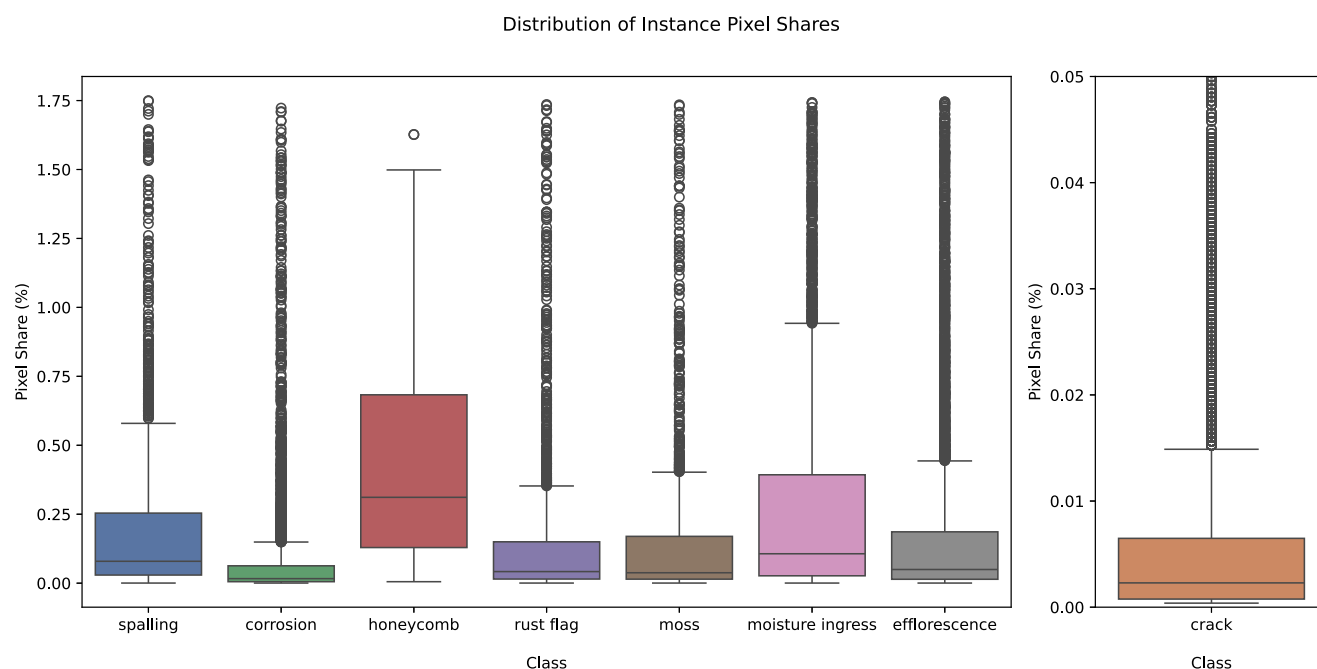


FIGURE 7 Boxplots showing the relative size distribution of damage masks for each class, calculated as the proportion of mask pixels to total image pixels.

the high representation in the data set mitigates the potential impact of small mask size. Other classes, such as moss and rust flag, are also challenging to detect as they not only have relatively small mask sizes (with median values of 0.04 %, respectively) but also appear less frequently in the data set (Figure 5). This is particularly disadvantageous for

rust flag due to its similarity with corrosion, which may affect classification performance.

Moisture ingress masks show relatively moderate sizes, with median and upper quartile values of 0.11 % and 0.39 %, respectively, while corrosion masks have the second smallest sizes, with a median of 0.02 %. Efflorescence



masks vary widely, with a median size of 0.05 % and an upper quartile of 0.19 %. Spalling masks, after honeycomb and moisture ingress masks, are comparatively larger than most other classes, with a median of 0.08 % and an upper quartile of 0.25 %.

The data distribution analysis demonstrates that the data set is imbalanced regarding damage class representation and damage mask size. While the high frequency of crack masks mitigates their small size, many other damage classes suffer from both small mask sizes and underrepresentation. This dual challenge can adversely affect model performance for less common damage types.

Additionally, the small mask sizes indicates that the images contain a significant amount of background information, increasing the difficulty of true positive detection. This can make the detection process more challenging due to a higher likelihood of false results. However, this can also increase the model's robustness by forcing it to distinguish subtle damage features from the background in application.

4.4 | Data augmentation

During training, we applied data augmentation using the Albumentations library. This included various operations to simulate real-world variability like horizontal and vertical flips ($p = .5$), random small-angle rotations (limit: $\pm 15^\circ$, $p = .5$), random resized crops, brightness and contrast shifts (limit: 0.2), Gaussian noise ($p = .2$), blur ($p = .2$), elastic deformation, grid distortion, and optical distortion (each with $p = .2$). Additionally, we included 93 complex background images in the training data set to increase the model's robustness against false positives in cluttered scenes.

4.5 | Multi-damage semantic segmentation network

Multi-class semantic segmentation in bridge inspection is challenging due to several factors: the large number of damage classes, high intra-class variability, instance similarity across classes, small object sizes (e.g., cracks covering less than 0.015 % of an image), and data imbalance across classes. To address these challenges, we adopted Feature Pyramid Network (FPN) as the semantic segmentation architecture, using an ImageNet-pretrained EfficientNet-B2 encoder which gave the best mean validation intersection-over-union (IoU) results among various standard semantic segmentation networks (e.g. U-Net, SegFormer, and DeeplabV3) in combination with various backbones (Iakubovskii, 2019). The network

TABLE 1 Per-class and mean validation precision, recall, and IoU results of the FPN model training.

	Precision	Recall	IoU
Spalling	35.1 ± 3.2	40.2 ± 2.9	23.8 ± 1.2
Crack	24.8 ± 2.6	41.2 ± 2.2	20.1 ± 1.3
Corrosion	44.0 ± 4.8	42.4 ± 5.1	26.8 ± 2.1
Honeycomb	7.1 ± 5.6	2.3 ± 2.2	2.1 ± 2.0
Rust flag	38.2 ± 4.1	45.8 ± 5.1	24.8 ± 2.1
Moss	34.1 ± 2.8	43.2 ± 8.5	23.8 ± 2.6
Moisture ingress	28.1 ± 4.3	39.7 ± 5.4	19.0 ± 2.3
Efflorescence	37.2 ± 2.3	43.2 ± 3.6	24.2 ± 1.3
Mean	31.1 ± 1.2	37.2 ± 1.4	20.6 ± 0.7

was trained with a composite loss function consisting of:

- (1) focal loss (weight 0.4) to handle class imbalance,
- (2) dice loss (weight 0.4), and
- (3) skeleton-recall loss (weight 0.2) (Kirchhoff et al., 2024) to improve recall of thin and fine-grained structures like cracks.

Pixel-wise class weights were computed from the label distribution and added into the loss function. Optimization was performed using the AdamW optimizer (learning rate: $1e-4$, batch size: 16). A fivefold cross-validation scheme was employed with early stopping based on mean validation IoU to prevent overfitting. Evaluation metrics included IoU, precision, and recall.

Additionally, we applied test-time augmentation to increase robustness. It involves flipping each input image horizontally, vertically, and in both directions, followed by averaging the prediction results. The results of this FPN training are shown in Table 1.

4.6 | Multi-damage segmentation results and discussion

The best performance was achieved with a mean IoU of 20.6 %. Recall was notably higher than precision (37.2 % vs. 31.1 %), indicating a higher rate of false positives than false negatives.

Among the individual damage classes, the highest IoU was observed for *corrosion* (26.8 %), followed by *rust flag* (24.8 %) and *efflorescence* (24.2 %). *Spalling* and *moss* followed closely with identical scores of 23.8 %. The *crack* class, despite a relatively large number of training samples, achieved an IoU of only 20.1 %. The lowest score was obtained for *honeycomb* (2.1 %), suggesting no meaningful learning and likely random predictions.



When comparing these results with data set characteristics visualized especially in Figures 5 and 7, no clear correlation was found between IoU performance and either pixel share or number of masks per class. Interestingly, the best-performing classes, *corrosion* and *rust flag*, have relatively low pixel shares. We hypothesize that their distinctive and consistent visual patterns, particularly in terms of color, contribute to higher model performance. In contrast, classes such as *moisture ingress* and *efflorescence* exhibit greater color variability and are often visually similar to the background, which may negatively impact segmentation accuracy.

During training, the FPN network, as with all tested architectures, showed signs of overfitting within the first five epochs. This is attributable to two primary factors: (1) the high visual complexity of the images, characterized additionally by class imbalance and a dominant background pixel share (see Figure 7), and (2) the limited data set size, which could not be fully compensated for through the applied data augmentation techniques.

Nevertheless, the model performs acceptably well on images featuring clearly visible, large, and simply shaped damage instances. Figure 8 illustrates representative segmentation results across various damage classes. With the exception of the honeycomb case (c), which was incorrectly segmented and misclassified as spalling and efflorescence, all other examples demonstrate successful segmentation and correct classification of the main damage instances. These include fully captured and correctly identified damages such as (a) crack, (b) efflorescence, (f) rust stain, and (g) spalling with corroded rebar, as well as partially segmented but correctly classified instances such as (d) moisture ingress and (e) moss.

Potential improvements include pretraining with copy-edit-paste strategies as proposed in Çelik & König (2022), and augmenting training with crops from original images of size 1024×1024 instead of 512×512 . Another possibility is to adopt a class-wise model training strategy. However, none of these techniques will yield enough performance increase. A data set with a minimum size in the 5-digit range is necessary, which exceeds the scope of this study.

Despite the limitations in segmentation performance, the model's integration within the 3D damage documentation system remains effective due to the system's semi-autonomous design. Users are able to review and correct model outputs, mitigating the impact of false predictions. To further reduce false positives, a conservative confidence threshold of 0.7 was applied during integration when binarizing the segmentation output. The next stage of postprocessing the segmentation results for structured documentation is discussed in Section 6.1.

5 | AUTOMATED CONDITION RATING WITH NLP CLASSIFICATION

Rating damage conditions is one of the most crucial steps in bridge management and inspection. It enables the evaluation of the overall bridge state and facilitates cost estimation for necessary repairs. In Germany, each damage instance is assessed based on three criteria: structural stability, traffic safety, and durability. For each criterion, a rating between 0 and 4 is assigned, where 0 indicates that the damage has no impact on the bridge with respect to the criterion, while 4 signifies that the criterion is entirely unfulfilled.

Bridge inspections are carried out by different inspectors, so it is essential to use standardized recording procedures to maintain consistency. To ensure uniformity, German authorities mandate the use of an official catalog containing over 1500 damage examples, each accompanied by a condition rating for the three criteria. When damage is detected, the most similar damage example from the damage catalog must be selected. The condition rating of the identified damage is then inferred from the rating of the chosen example, with potential adjustments within a margin of one score based on factors such as size, extent, and precise location of the damage.

ML is the ideal solution for the automation of damage rating that can process voluminous labeled structural damage data stored in official state databases. It can effectively capture the complexity of damage data by identifying patterns and correlations within the condition ratings, thereby reflecting the collective knowledge of numerous bridge inspectors.

We evaluated the feasibility of this approach using data from the German BMS *SIB-BAUWERKE*. This database is comprehensive, documenting all structural damage observed on federal and state roads and highways in Germany, including bridges and other structures. It contains textual descriptions and images of damage.

For the ML task, we focused exclusively on textual data, excluding images, for three main reasons: First, textual descriptions already contain comprehensive information about the damage, such as the bridge type, component affected, damage type, size, quantity, and additional details conveyed through inspectors' comments. These free-text annotations provide nuanced and context-rich information that cannot be reliably inferred from images alone. Second, processing and training on textual data is more efficient and environmentally sustainable, as it requires less energy and memory compared to handling large volumes of image data. Third, the image data stored in the database are compressed and of suboptimal quality. This makes them unsuitable for large-scale annotation.

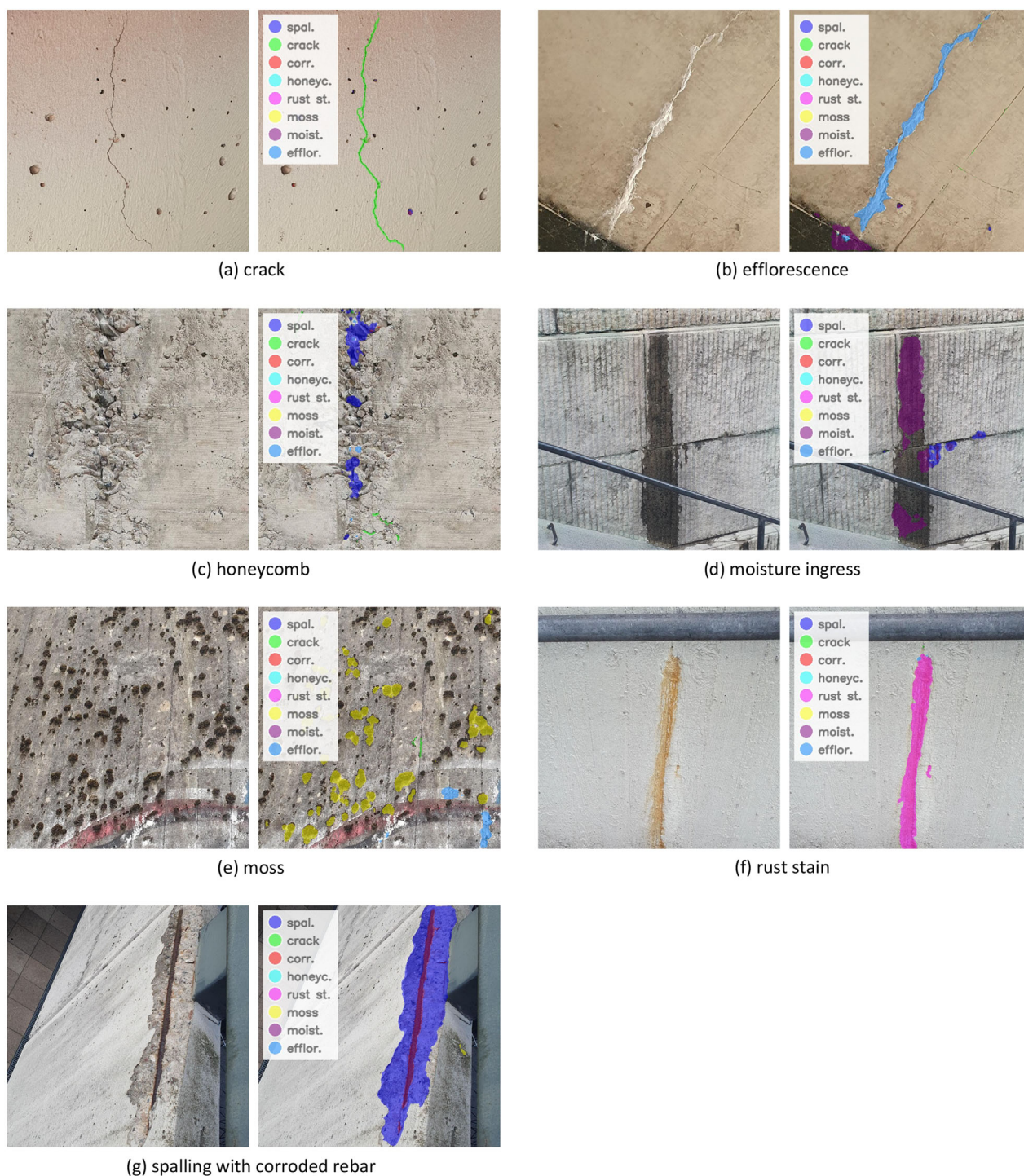


FIGURE 8 Representative examples of damage segmentation results for different damage classes: (a) crack, (b) efflorescence, (c) honeycomb, (d) moisture ingress, (e) moss, (f) rust stain, and (g) spalling with corroded rebar. For each pair, the right image displays the corresponding segmentation output with overlaid damage masks.

5.1 | Data preprocessing and preparation

The data to be used for the ML-based condition rating are the same CSV data introduced in Subsection 4.1. The data are structured in a tabular format and contains 2,237,628 samples, each representing an individual

structural damage record. Each data sample comprises 37 columns containing various attributes, ranging from structural component details to damage ID. However, only 11 columns are relevant for ML-based damage rating: *component* (Bauteil), *structural element* (Konstruktionsteil), *component extension* (Bauteilergänzung), *component group*



Original data sample in German:

```
"Wand ";NULL;NULL;"0,2 - < 0,4 mm "; "Mehrfach
";"xxx Stück ";NULL;NULL;"
";NULL;NULL;NULL;1;0;0;NULL;NULL;"
";NULL;"AS Musterstadt ";1999;"H ";stellenweise bis
0,6 mm, vereinzelt in Schrägrisse übergehend;" ";1;0;0;"
";" "; "Bauteilgruppe Unterbau ";5718620;" 2";"0781172
";"5710740 3 ";8S0P1YKT;"A85 /
Tim.Str.""Musterstraße""/A85 /
Tim.Str.""Musterstraße"", FR
Musterstadt";"Plattenbrücke"
```

Processed final sample in German:

```
input_text: "unterbau wand, riss, 0,2 - <
0,4 mm, stellenweise bis 0,6 mm,
vereinzelt in schrägrisse übergehend, 1"
output: "1, 0, 0"
```

English translation of the *input_text* sample:

```
input_text: "substructure wall, crack, 0.2
- < 0.4 mm, in some spots up to 0.6 mm,
occasionally transitioning into diagonal
cracks, 1"
```

FIGURE 9 Example of preprocessing a noisy German bridge inspection entry. The original raw data (top), derived from 37 input columns, has been anonymized to protect data privacy by modifying IDs and the official structure name. The preprocessed version (middle) shows the normalized text used for model training. For reference, an English translation of the German *input_text* is provided (bottom). This example contains no entries for *structural element* or *component extension*, but includes information for *component group* (substructure) and *component* (wall). The trailing “1” in the *input_text* denotes the general quantity type medium (see in the [Appendix](#) for details on general quantity coding).

(Bauteilgruppe), *general damage quantity*, (*free-text*, *damage type*, *damage progress*, *structural stability*, *traffic safety*, and *durability*. So, first we removed the 26 columns that are irrelevant for condition rating.

The most important input feature is *damage type*. As described in Subsection 4.1, we defined eight general damage classes that subsume a total of 22 specific damage types listed in *SIB-Bauwerke*. Only data samples corresponding to these damage types were included for further analysis and ML training. Then we applied multiple preprocessing steps to prepare the data as training data. The details of the preprocessing can be read in the [Appendix](#). Figure 9 shows one original input sample and the equivalent processed sample in its final form. After the application of all filters described in this section and in the [Appendix](#), the initial number of 2,237,628 damage samples reduced to 172,496.

5.2 | Data distribution

The following presents the data distribution for the BMS data set, with a primary focus on the output data, that is, the rating labels. Additionally, the distribution of damage classes within the processed BMS data is considered. Among the 172,496 damage samples, 41.48 % belong to *crack*, 21.44 % to *spalling*, 20.71 % to *spalling with exposed rebar*, 3.91 % to *honeycomb*, 3.79 % to *efflorescence*, 2.92 % to *moisture ingress*, 2.72 % to *moss*, 1.65 % to *honeycomb with exposed rebar*, and 1.39 % to *rust flag*. The data distribution shows a clear overrepresentation of cracks, which account for over 40 % of all samples, which is consistent with trends observed in the image data set. Crack-related data are followed by spalling and spalling with exposed rebar, which together contribute a similarly large portion. The remaining damage classes are substantially underrepresented, each comprising less than 4 % of the data set.

Next, we analyzed the *output* data, which consist of three values that each represent the rating for the condition criteria durability, traffic safety, and structural stability. The ratings range from 0 (indicating that the damage does not affect the criterion) to 4 (indicating that the damage no longer fulfills the criterion).

Figure 10 illustrates the class distribution for each condition criterion. For durability, the distribution is relatively balanced between class 1 (51.00 %) and class 2 (42.07 %), with class 0 representing 2.71 %, class 3 4.20 %, and class 4 a negligible 0.02 %. In contrast, traffic safety is heavily skewed toward class 0, which accounts for 92.89 %, followed by class 1 (5.15 %), class 2 (1.73 %), class 3 (0.23 %), and class 4 (0.00 %). A similar pattern is observed for structural stability, where class 0 dominates with 90.51 %, followed by class 1 (8.30 %), class 2 (1.02 %), class 3 (0.17 %), and class 4 (0.00 %).

These distributions highlight a significant class imbalance, particularly for traffic safety and structural stability, where the vast majority of samples fall into class 0. Notably, class 4 is entirely absent in both of these criteria, and is marginally available in durability. In Germany, a rating of 4 in any condition criterion mandates the immediate closure of the bridge. As a result, such cases are often repaired or removed from service before they are documented in subsequent inspections, which leads to a survivor bias (Marsili et al., 2025). Due to this data situation, samples with a rating of 4 in durability are excluded from training. This means that the NLP models will not be able to predict ratings of 4. However, this does not make the models necessarily incomplete. From the perspective of responsible AI design in safety-critical domains, the exclusion of rating 4 can be interpreted as a deliberate safety mechanism. Rating a damage with score

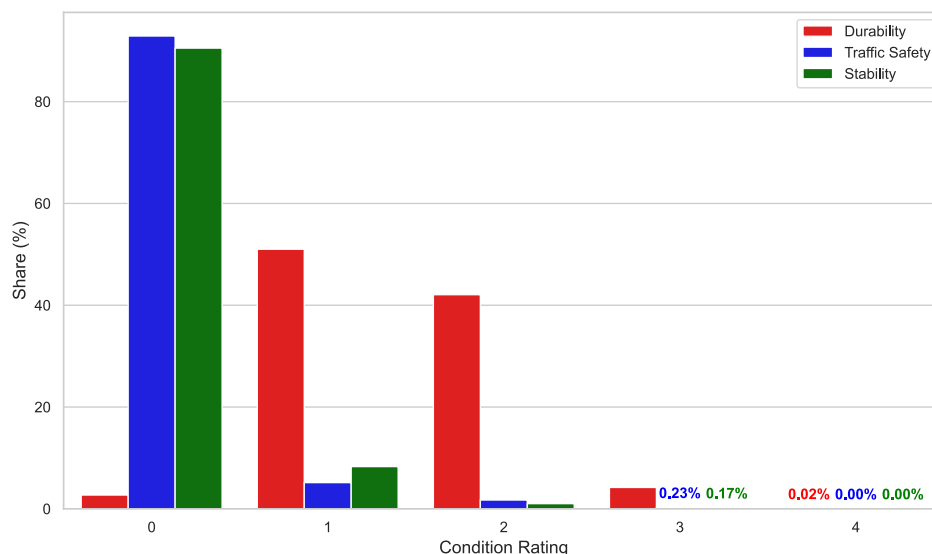


FIGURE 10 The share of rating classes for each condition criterion (durability, traffic safety, structural stability). The bars represent the percentage distribution across rating classes from 0 to 4 for each criterion. The annotations on the right-hand side highlight low-frequency classes (ratings of 3 and 4) as they are not visible.

4 carry significant operational and legal consequences and is therefore reserved for human experts.

5.3 | Data tokenization and LSTM classification training for condition rating

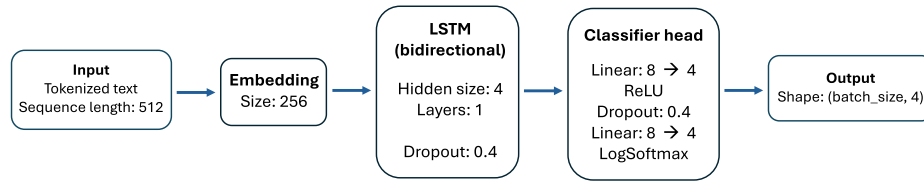
The ML task is to use *input_text* as input data and *output* as label data to train NLP-based classification networks that predict the condition rating for structural stability, traffic safety, and durability, respectively. When analyzing the *input_text* data, it becomes obvious that the text samples are relatively short and have a consistent pattern: damage location description, damage type and description, and damage quantity. Preliminary experiments with Transformer-based models, such as BERT, exhibited unstable training behavior. Consequently, we adopted lightweight NLP architectures, specifically LSTMs, to train the damage rating models.

Before training, the input data have to be tokenized. Since the data contain domain-specific bridge terminology, we built a new tokenizer instead of using general off-the-shelf German tokenizers. To this end, we used the WordPiece model, which breaks words into subword units after splitting the text on spaces with a Whitespace pre-tokenizer. The most important parameter in the WordPiece model is the vocabulary size. If too large a size is chosen, then we might have too many parameters for the embedding layer complicating the training of the LSTM networks. If too small a size is chosen, then the vocabulary may be overly fragmented. We tested vocabulary sizes between 18,000 and 40,000 and found vocabulary size 25,000 as the

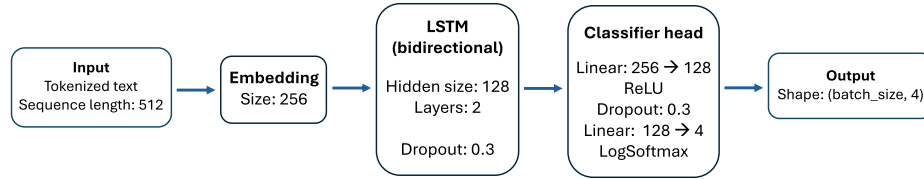
most optimal for our data. We used the python library *tokenizers* for the implementation of both tokenizer models Whitespace and WordPiece.

In the next step, we built and tested multiple LSTM models with various LSTM module connections for each rating criterion. For durability rating, the architecture with the most accurate results is a one-layer bidirectional LSTM network (Graves et al., 2005). The full model architecture consists of a 256-dimensional word embedding layer followed by a one-layer bidirectional LSTM with 4 hidden units per direction and application of dropout with a rate of 0.4. The final hidden states from both directions are concatenated, resulting in an 8-dimensional sequence representation. This is passed through batch normalization and finally to a feedforward classifier consisting of following layers: Linear (8, 4) → ReLU → Dropout (0.4) → Linear (4, 4) → LogSoftmax. The final layer outputs an array with a dimension of 4 representing ratings of 0 to 3. The model architecture for durability is visualized in Figure 11a).

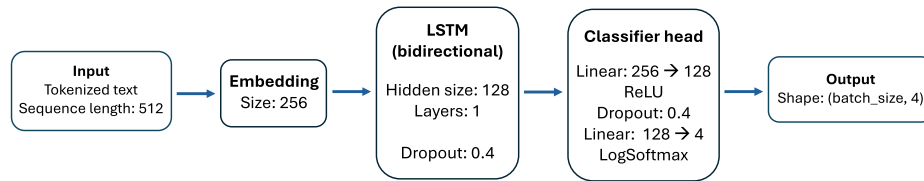
The network was trained with batch size 128 and Adam optimizer with learning rate $1e-3$ and weight decay $1e-4$. Additionally, a learning rate scheduler (*ReduceLROnPlateau*) was used to adapt the learning rate dynamically based on validation macro-F1 score performance and early stopping was implemented. Next to dropout, we applied three strategies to handle class imbalance. One is balanced sampling, that is, we sampled equal number of instances from each class. We tested multiple sample sizes and found sample size 20,000 to be optimal. If a class had fewer samples, we selected the samples more than once until the defined sample size was reached. Also, we defined a custom loss function, which is a sum of



(a) LSTM-Network Architecture for Durability



(b) LSTM-Network Architecture for Traffic Safety



(c) LSTM-Network Architecture for Structural Stability

FIGURE 11 Overview of the LSTM-based network architectures used for predicting durability, traffic safety, and structural stability based on German bridge inspection text. Each architecture differs in terms of LSTM depth, hidden size, and dropout rate.

two components. The first is a weighted cross-entropy loss, where class weights are used to emphasize under-represented classes. These weights were computed before applying balanced data sampling. The second component is a distance-based penalty term that calculates the absolute difference between the predicted class and the ground truth. We introduced this penalty term since we have an ordinal classification problem in which the degree of misprediction is important, that is, a prediction of 3 instead of 0, is a larger error than predicting 1 instead of 0. The full loss function L is defined as

$$L = \frac{1}{N} \left[\underbrace{- \sum_{i=1}^N w y_i \log(\hat{y}_i)}_{\text{CE-Loss}} + \underbrace{\sum_{i=1}^N |\text{argmax}(y_i) - \text{argmax}(\hat{y}_i)|}_{\text{distance-based loss}} \right]$$

where N is the batch size, w the class weights, $\hat{y}_i$ the predictions, and y_i the ground truth labels.

For traffic safety, the architecture with the most accurate results is a two-layer bidirectional LSTM network. The full model architecture consists of a 256-dimensional word embedding layer followed by two layers of bidirectional LSTM with significantly higher hidden units per direction

($\rightarrow 128$) than in the network for durability ($\rightarrow 4$). In both LSTM layers we applied also dropout with a rate of 0.3. The output from the final LSTM layers is passed through a feed-forward classifier without applying batch normalization as in the durability network. The classifier consists of the following layers: Linear (256, 128) $\rightarrow$ ReLU $\rightarrow$ Dropout (0.3) $\rightarrow$ Linear (128, 4) $\rightarrow$ LogSoftmax.

The traffic safety network was trained with a larger batch size, 512, and Adam optimizer with learning rate $1e-3$ but without weight decay. Other training configuration includes a learning rate scheduler (*ReduceLROnPlateau*) based on validation macro-F1 score performance, early stopping, and gradient clipping with a maximum norm of 1.0 to prevent exploding gradients. This network architecture is visualized in Figure 11b).

We applied the same strategies to mitigate the effects of class imbalance during training like for the durability network. The only difference was that we set the sample size to 30,000 for sample balancing, which we found to be the most optimal size regarding network performance.

The network for structural stability is a one-layer bidirectional LSTM network. The model architecture consists of a 256-dimensional word embedding layer followed by a one-layer bidirectional LSTM with 128 hidden units and a dropout-layer with a rate of 0.4. The LSTM output is



TABLE 2 Durability prediction: Macro and per-class performance across Train, Validation, and Test sets.

Class	Metric	Train	Validation	Test
Macro	Precision	72.0 ± 3.9	71.5 ± 1.6	72.0 ± 1.3
	Recall	69.6 ± 3.8	72.1 ± 1.7	72.6 ± 1.2
	F1-score	69.5 ± 3.6	71.5 ± 1.7	72.0 ± 1.3
0	Precision	85.1 ± 6.0	85.5 ± 3.1	86.3 ± 2.1
	Recall	80.6 ± 3.4	92.0 ± 2.1	92.9 ± 1.9
	F1-score	82.7 ± 3.5	88.6 ± 1.0	89.4 ± 0.8
1	Precision	67.5 ± 9.4	63.7 ± 3.5	64.0 ± 2.7
	Recall	57.0 ± 15.7	54.8 ± 5.5	55.2 ± 5.7
	F1-score	60.0 ± 4.7	58.7 ± 2.7	59.0 ± 2.6
2	Precision	58.8 ± 12.7	57.2 ± 3.3	57.7 ± 2.4
	Recall	58.3 ± 4.9	55.1 ± 5.0	55.1 ± 3.9
	F1-score	57.6 ± 4.7	55.9 ± 2.7	56.3 ± 2.2
3	Precision	76.7 ± 12.1	79.6 ± 4.3	79.9 ± 4.0
	Recall	82.3 ± 13.2	86.4 ± 6.6	87.0 ± 5.5
	F1-score	77.9 ± 3.0	82.6 ± 1.7	83.1 ± 1.2

passed through a feedforward classifier, without applying batch normalization, which consists of the following layers: Linear (256, 128) $\rightarrow$ ReLU $\rightarrow$ Dropout (0.4) $\rightarrow$ Linear (128, 4) $\rightarrow$ LogSoftmax. The training configuration as well as the strategies for handling data imbalance are the same as for the traffic safety network. The only difference is that the optimal sample size for balanced sampling is 35,000 samples. This network architecture is visualized in Figure 11c.

5.4 | Condition rating network results and discussion

We applied fivefold cross-validation to the data set, reserving 10% as a separate test set. Model performance was evaluated using precision, recall, and F1-score. Tables 2–4 summarize macro-level and per-class performance across training, validation, and test splits for all three condition criteria: durability, traffic safety, and structural stability.

Among the three models, the durability network yielded the lowest performance, with a macro F1-score of 71.5%. A closer inspection reveals notable differences across individual classes. Specifically, the model performs substantially better on classes 0 and 3, achieving validation F1-scores of 88.6% and 82.6%, respectively, compared to 58.7% and 55.9% for classes 1 and 2. This 28.3-point difference indicates that the applied class balancing strategy may have been overly aggressive or suboptimal. Interestingly, class 0, despite being less represented than class 3 in the original data (cf. Figure 10), achieved a higher F1-score, suggesting that the balancing did not translate proportion-

TABLE 3 Traffic safety prediction: Macro and per-class performance across Train, Validation, and Test sets.

Class	Metric	Train	Validation	Test
Macro	Precision	99.9 ± 0.1	96.6 ± 0.2	96.5 ± 0.1
	Recall	99.9 ± 0.1	96.5 ± 0.2	96.4 ± 0.1
	F1-score	99.9 ± 0.1	96.5 ± 0.2	96.4 ± 0.1
0	Precision	99.9 ± 0.1	97.4 ± 0.2	97.3 ± 0.1
	Recall	99.8 ± 0.2	89.4 ± 0.5	89.3 ± 0.1
	F1-score	99.8 ± 0.2	93.2 ± 0.3	93.1 ± 0.1
1	Precision	99.8 ± 0.2	91.9 ± 0.4	91.8 ± 0.3
	Recall	99.9 ± 0.1	96.7 ± 0.3	96.5 ± 0.3
	F1-score	99.8 ± 0.2	94.2 ± 0.3	94.1 ± 0.2
2	Precision	99.9 ± 0.1	97.3 ± 0.4	97.2 ± 0.2
	Recall	100.0 ± 0.0	99.9 ± 0.1	99.9 ± 0.0
	F1-score	99.9 ± 0.1	98.6 ± 0.2	98.5 ± 0.1
3	Precision	100.0 ± 0.0	99.7 ± 0.1	99.8 ± 0.1
	Recall	100.0 ± 0.0	100.0 ± 0.0	100.0 ± 0.0
	F1-score	100.0 ± 0.0	99.9 ± 0.0	99.9 ± 0.0

TABLE 4 Structural stability prediction: Macro and per-class performance across Train, Validation, and Test sets.

Class	Metric	Train	Validation	Test
Macro	Precision	99.9 ± 0.1	95.4 ± 0.3	95.4 ± 0.2
	Recall	99.9 ± 0.1	95.3 ± 0.3	95.3 ± 0.2
	F1-score	99.9 ± 0.1	95.3 ± 0.3	95.3 ± 0.2
0	Precision	99.9 ± 0.1	94.4 ± 0.5	94.7 ± 0.1
	Recall	99.8 ± 0.2	87.1 ± 1.1	86.8 ± 0.7
	F1-score	99.9 ± 0.1	90.6 ± 0.6	90.6 ± 0.4
1	Precision	99.8 ± 0.2	88.5 ± 0.8	88.5 ± 0.6
	Recall	99.9 ± 0.1	94.1 ± 0.6	94.5 ± 0.1
	F1-score	99.8 ± 0.1	91.2 ± 0.5	91.4 ± 0.3
2	Precision	100.0 ± 0.0	98.9 ± 0.2	98.7 ± 0.1
	Recall	100.0 ± 0.0	100.0 ± 0.0	99.9 ± 0.0
	F1-score	100.0 ± 0.0	99.4 ± 0.1	99.3 ± 0.1
3	Precision	100.0 ± 0.0	99.8 ± 0.1	99.8 ± 0.0
	Recall	100.0 ± 0.0	100.0 ± 0.0	100.0 ± 0.0
	F1-score	100.0 ± 0.0	99.9 ± 0.0	99.9 ± 0.0

ally into generalization performance. Also, the results for classes 1 and 2 show signs of overfitting, whereas classes 0 and 3 exhibit stronger generalization to the validation and test sets.

From an application standpoint, poor performance on classes 1 and 2 is particularly problematic, as the majority of damage cases in bridge inspections fall into these intermediate severity ratings. The confusion matrix in Figure 12a shows true positive rates of 61.0% for class 1 and 63.0% for class 2. Notably, most misclassifications are off by only one score: class 1 is most often misclassified as class

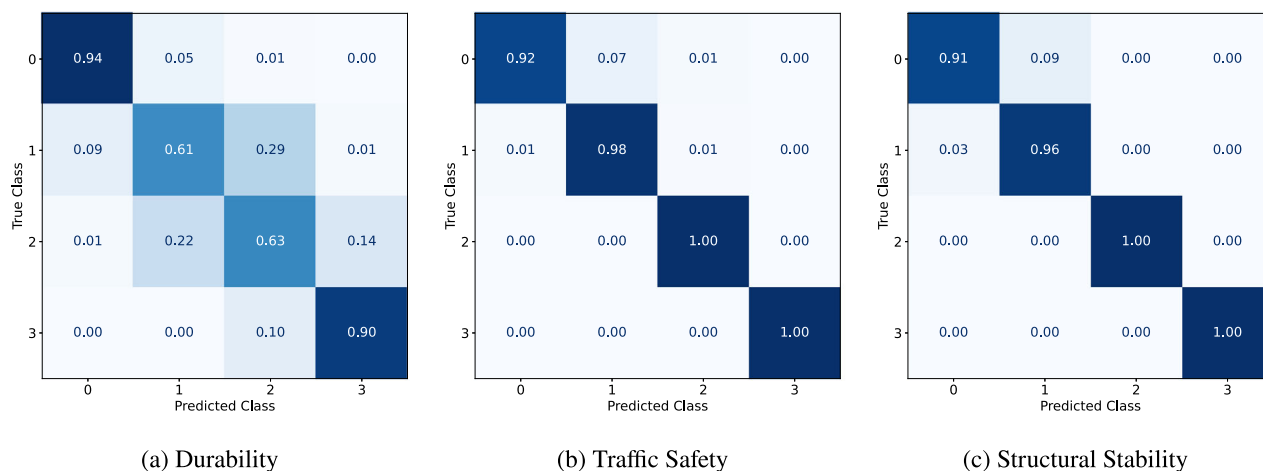


FIGURE 12 Normalized confusion matrices for all three LSTM-based condition classifiers. The data show the average results on the test data set of all five models from fivefold cross-validation.

2 (29.0 %), and class 2 as class 1 (22.0 %). While such near-miss errors are less critical than larger deviations, they still impact operational decision making.

In contrast, the traffic safety and structural stability models demonstrate consistently high performance and similar behavior. The macro F1-scores for these tasks are 96.5 % and 95.3 %, respectively. Validation F1-scores across all classes range from 93.2 % to 99.9 % for traffic safety, and 90.6 % to 99.9 % for structural stability. Although slight overfitting is evident, with training F1-scores reaching 99 % or higher, the near parity between validation and test results indicates strong robustness.

The confusion matrices in Figure 12b and c confirm this pattern. For traffic safety, class 0 achieves a true positive rate of 92.0 %, and class 1 reaches 98.0 %. For structural stability, these values are 91.0 % and 96.0 %, respectively. Errors are predominantly misclassifications by one score: class 0 is most often confused with class 1 (7.0 % in traffic safety and 9.0 % in stability), whereas the opposite, class 1 being misclassified as class 0, occurs less often (1.0 % and 3.0 %, respectively). Importantly, there are no misclassifications for classes 2 and 3 in either criterion, further highlighting the models' reliability.

In summary, the LSTM model for durability underperforms in comparison to the highly robust models developed for traffic safety and structural stability. This performance gap is likely attributable to a combination of model capacity limitations, data complexity, and the impact of the class balancing strategy. For example, due to undersampling, only 22.0 % and 27.0 % of the available training samples were retained for classes 1 and 2 in the durability data set. Likewise, for class 0 in traffic safety and structural stability, only 22.0 % and 18.0 % of the data were used, respectively. Conversely, other classes were oversampled by duplicating entries without any data aug-

mentation. This underscores the need for more representative and balanced data, particularly for underrepresented classes, to avoid the pitfalls of data loss through undersampling and overfitting due to oversampling. Nonetheless, the results demonstrate that the proposed preprocessing pipeline and the use of compact LSTM architectures can yield strong predictive performance.

From a deployment perspective, integrating these models into operational bridge inspection systems must involve appropriate human-in-the-loop mechanisms. Specifically, the system should issue advisory prompts when predictions fall into critical ranges, such as ratings of 1 or 2 for durability and ratings of 0 or 1 for traffic safety and structural stability, to ensure that these cases are manually reviewed by qualified inspectors. Such safeguards are essential to maintain accountability, especially in contexts where ML outputs influence infrastructure decisions.

6 | INTEGRATION OF DAMAGE SEGMENTATION AND RATING IN THE SYSTEM

6.1 | Postprocessing of damage segmentation results

Structured documentation of detected damage is essential for reliable automated condition rating. The defined standardized text input format for the LSTM models consists of: (1) damage location, (2) damage type, (3) damage description (free-text), and (4) damage quantity.

The input generation regarding damage type requires special attention. In practice, a damage image may contain multiple overlapping or co-occurring damage types. It is therefore necessary to distinguish between primary and

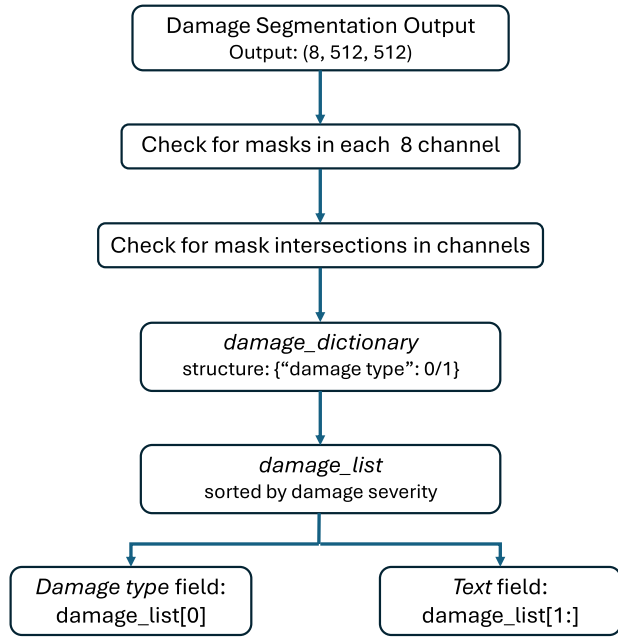


FIGURE 13 Workflow illustrating each step in the postprocessing of the damage segmentation output, from raw mask analysis to structured field entry in the information panel.

secondary damages. The primary damage is assigned to the main damage type field, while any secondary damage types are recorded in the free-text description field. To support this workflow, the system applies a three-step image analysis following image upload: (1) damage segmentation, (2) damage presence check, and (3) damage hierarchization. A schematic overview of these analysis steps is shown in Figure 13.

In the first step, the uploaded image is passed through the multi-class semantic segmentation model, which returns an output tensor of shape (8, 512, 512) representing eight binary masks, one per damage class. In the second step, the system initializes a structured dictionary of binary presence indicators, corresponding to each of the eight base damage classes. Each mask is checked for nonzero entries to determine the presence of the respective class. In addition to detecting base classes, the system also infers compound damage types by evaluating the pixel-wise intersections of relevant masks. Specifically, the following compound types are derived using set intersection, where M_{class} denotes the binary segmentation mask for a given damage class:

- (1) *spalling with exposed rebar*: $M_{\text{spalling}} \cap M_{\text{corrosion}}$
- (2) *honeycomb with exposed rebar*: $M_{\text{honeycomb}} \cap M_{\text{corrosion}}$
- (3) *crack with moisture ingress and rust flag*: $M_{\text{crack}} \cap M_{\text{moisture ingress}} \cap M_{\text{rust flag}}$
- (4) *crack with rust flag*: $M_{\text{crack}} \cap M_{\text{rust flag}}$

- (5) *moisture ingress with efflorescence*: $M_{\text{moisture ingress}} \cap M_{\text{efflorescence}}$

The identified damage types, both base and compound, are compiled into a candidate list. This list is then reordered based on a predefined damage severity hierarchy, which reflects domain knowledge regarding structural impact. Damage types higher in the hierarchy are considered more severe. The applied hierarchy is as follows:

- (1) *spalling with exposed rebar*
- (2) *honeycomb with exposed rebar*
- (3) *crack with moisture ingress and rust flag (rust color)*
- (4) *crack with rust flag*
- (5) *crack*
- (6) *efflorescence*
- (7) *spalling*
- (8) *honeycomb*
- (9) *rust flag*
- (10) *moisture ingress with efflorescence*
- (11) *moisture ingress*
- (12) *moss*

Finally, the most severe detected damage type is automatically selected as the primary damage and recorded in the dedicated field of the 3D viewer interface. All secondary or co-occurring damage types are listed in the free-text description field, allowing users to provide additional context or verification during review.

Our full image processing pipeline, including preprocessing and postprocessing, achieves an average runtime of 116.40 ms per image on an Nvidia GeForce RTX 4060, corresponding to real-time operation (8.6 FPS). The segmentation model alone achieves an inference time of 86.35 ms per image. While this setup was evaluated on desktop hardware, the model's moderate computational footprint render deployment on mobile or edge devices feasible with further optimization, such as quantization.

6.2 | Damage input preparation for damage rating

Once a damage annotation is created in the 3D bridge model interface, relevant location metadata, such as component, structural element, component extension, and component group, is automatically retrieved and stored in the backend's damage instance dictionary. Additional information, including the selected damage type, free-text, and damage quantity, is also appended to this dictionary.

When the user initiates the rating process by clicking the *Rating* button, the system compiles all stored dam-



age information into a structured input string. This string concatenates location descriptors, damage type labels, and quantity values into a standardized format compatible with the LSTM-based condition rating models, as illustrated in Figure 9. The resulting string is then tokenized and passed to the condition rating models.

7 | DISCUSSION

The system was tested using example damage images and text samples from the German BMS test data set (see Section 5.4). Technically, all components functioned as intended, including the integration of damage segmentation and LSTM-based damage rating. The end-to-end workflow, from annotating a damage location in the 3D viewer to obtaining automated condition ratings, produced mostly reliable predictions. Even when introducing spelling variations or uncommon abbreviations not seen during training, the damage rating model maintained a reasonable level of robustness.

The primary source of error was the semantic segmentation output, which occasionally produced false positives or missed smaller damage regions. A comprehensive evaluation of these errors is left for future work, particularly once a more balanced and representative training data set becomes available. Another current limitation lies in the segmentation model's label space, which supports only 12 damage types, despite being one of the most extensive proposed in the literature. While these include the most frequently documented categories in *SIB-Bauwerke*, many other damage types are not detectable. Furthermore, the practical labeling approach requires further validation to account for unexpected combinations in real-world scenarios. For example, we encountered the following edge cases:

- (1) A spalling region containing corroded metallic objects such as nails or wires, which the system classified as *spalling with exposed rebar* even though no rebar was present.
- (2) A case of concrete spalling with an exposed but non-corroded rebar, where the system detected *spalling* but failed to recognize the exposed rebar due to its uncorroded appearance.

Although rare, such exceptions reveal the limitations of the current label space in enabling fully autonomous interpretation of visual damage. Expanding the label set to include additional fine-grained classes (e.g., “exposed rebar”) could improve precision, but would also require considerable annotation effort and additional training data. Alternatively, ML approaches, such as vision-

language models, offer a promising direction for addressing this challenge, as many BMSs provide image–text pairs (Chun et al., 2024, 2022; Kunlamai et al., 2024).

Despite these limitations, systems with interactive interfaces, such as the one proposed here, are especially well suited for documenting rare or ambiguous damage cases. By allowing users to review and correct automated outputs in real time, the system serves as a practical bridge between manual inspection workflows and fully autonomous UAV-based damage documentation. Although currently deployed as a desktop software tool, the system conceptually simulates a mobile application that could be used on-site during bridge inspections. Such a semi-autonomous system not only assists inspectors, but also creates valuable training data for future iterations of fully automated ML models. By systematically collecting failure cases and exceptions, it can guide the evolution toward more robust, scalable inspection solutions.

8 | CONCLUSION

This research addressed a critical gap in digitalized bridge inspection: the integration of multi-damage detection, condition rating, and 3D structural context into a unified, ML-driven workflow. While many prior studies have explored isolated tasks such as crack detection and UAV-based damage localization, few have systematically linked visual detection outputs with standardized condition assessments required in real-world inspections. By grounding our work in German inspection standards while maintaining architectural flexibility, we propose a transferable system design compatible with diverse national regulations and BMSs.

We contributed three key advances toward practical inspection automation. First, we introduced a statistically and semantically derived damage taxonomy based on 2.2 million BMS records. It establishes a systematic foundation for multi-damage detection that explicitly supports downstream condition rating tasks. This taxonomy addresses the fundamental challenge of damage class definition by balancing coverage with practical annotation feasibility while ensuring compatibility with established inspection standards.

Second, we demonstrated that high-performing condition rating models can be trained solely from historical textual records. Our custom preprocessing pipeline and bidirectional LSTM architectures achieved up to 99% F1-score, without requiring paired image–text data, thus enabling the reuse of legacy BMS data and minimizing memory and computational demands.

Third, we implemented these components within an interactive 3D inspection system that enables a struc-



tured and traceable data flow from visual damage input to standardized condition output, embedding all steps within a cohesive human-in-the-loop workflow. Crucially, the system allows manual correction of ML outputs at every stage, for example, adjusting damage type output or modifying predicted condition ratings, and supports the addition of inspection-relevant information such as damage extent or progression. This design ensures robustness in real-world use.

Despite these advances, several limitations remain. The current system requires manual input for damage quantification and progress, such that full automation is not yet achieved. Our segmentation model is also limited to eight core damage classes and cannot yet generalize to abstract, rare, or emerging types. Moreover, the condition rating models, though trained on a large corpus, may not capture damage patterns associated with new materials (e.g., carbon reinforced concrete) or changing environmental conditions. However, the models can be finetuned and adapted as needed.

Building on this foundation, future research should explore automated quantification, textual description generation, and temporal damage tracking. Vision-language models and anomaly detection techniques offer promising paths for reducing annotation effort while improving interpretability and damage coverage. Active learning loops, supported by inspector annotation feedback via the interface, could drive continual model refinement. Additionally, integrating further data modalities, such as sensor or climate data, may enrich condition assessment.

This work demonstrates that the integration of visual detection and condition rating is not only a technical problem, but a systems-level challenge requiring alignment across damage taxonomy, model input-and-output requirements, and workflow integration. By connecting computer vision and NLP within a standards-aligned inspection framework, we offer a robust foundation for practical ML-assisted inspections. Our results suggest that future progress in this field will rely not only on individual model improvements but also on holistic coordination within scalable, inspector-centric systems.

ACKNOWLEDGMENTS

We would like to express our gratitude to the German federal road administration Straßen.NRW for providing access to data from the German BMS *SIB-Bauwerke*, and to the company Kempen Krause Ingenieure GmbH for supplying damage images used in training the damage segmentation models. We also thank Professor Martin Mertens (Bochum University of Applied Sciences), a certified bridge inspector, for his valuable input on bridge damage and for reviewing an early version of this paper. Computational resources were provided by the German AI

Service Center WestAI. This research is partially funded by dtec.bw — Digitalization and Technology Research Center of the Bundeswehr, which we gratefully acknowledge. dtec.bw is funded by the European Union – NextGenerationEU.

Open access funding enabled and organized by Projekt DEAL.

REFERENCES

- Abdallah, A. M., Atadero, R. A., & Özbek, M. E. (2022). A state-of-the-art review of bridge inspection planning: Current situation and future needs. *Journal of Bridge Engineering*, 27(2), 03121001. [https://doi.org/10.1061/\(ASCE\)BE.1943-5592.0001812](https://doi.org/10.1061/(ASCE)BE.1943-5592.0001812)
- Artus, M., Alabassy, M. S. H., & Koch, C. (2022). A BIM based framework for damage segmentation, modeling, and visualization using IFC. *Applied Sciences*, 12(6), 2772.
- Assaad, R., & El-Adaway, I. H. (2020). Bridge infrastructure asset management system: Comparative computational machine learning approach for evaluating and predicting deck deterioration conditions. *Journal of Infrastructure Systems*, 26(3), 04020032. [https://doi.org/10.1061/\(ASCE\)IS.1943-555X.0000572](https://doi.org/10.1061/(ASCE)IS.1943-555X.0000572)
- Azimi, M., & Yang, T. (2024). Transformer-based framework for accurate segmentation of high-resolution images in structural health monitoring. *Computer-Aided Civil and Infrastructure Engineering*, 39(24), 3670–3684.
- Benz, C., & Rodehorst, V. (2022). Image-based detection of structural defects using hierarchical multi-scale attention. In Andres, B., Bernard, F., Cremers, D., Frintrop, S., Goldlücke, B. & Ihrke, I. (Eds.), *Pattern recognition: 44th DAGM German Conference, DAGM GCPR 2022, Konstanz, Germany, September 27-30, 2022, Proceedings (Lecture Notes in Computer Science, Vol. 13485, pp. 337–353)*. Cham: Springer.
- Brighenti, F., Caspani, V. F., Costa, G., Giordano, P. F., Limongelli, M. P., & Zonta, D. (2024). Bridge management systems: A review on current practice in a digitizing world. *Engineering Structures*, 321, Article 118971. <https://doi.org/10.1016/j.engstruct.2024.118971>
- Çelik, F., Herbers, P., & König, M. (2022). Image segmentation on concrete damage for augmented reality supported inspection tasks. In Skatulla, S. & Beusheusen, H. (Eds.), *Advances in Information Technology in Civil and Building Engineering. ICCBE 2022. Lecture Notes in Civil Engineering*, vol 357, pp. 237–252. Cham: Springer.
- Çelik, F., & König, M. (2022). A sigmoid-optimized encoder-decoder network for crack segmentation with copy-edit-paste transfer learning. *Computer-Aided Civil and Infrastructure Engineering*, 37(14), 1875–1890.
- Cesare, M. A., Santamarina, J. C., Turkstra, C., & Vanmarcke, E. H. (1992). Modeling bridge deterioration with Markov chains. *Journal of Transportation Engineering*, 118(6), 820–833. [https://doi.org/10.1061/\(ASCE\)0733-947X\(1992\)118:6\(820\)](https://doi.org/10.1061/(ASCE)0733-947X(1992)118:6(820))
- Chen, J., & He, Y. (2022). A novel u-shaped encoder-decoder network with attention mechanism for detection and evaluation of road cracks at pixel level. *Computer-Aided Civil and Infrastructure Engineering*, 37(13), 1721–1736.
- Chun, P.-j., Chu, H., Shitara, K., Yamane, T., & Maemura, Y. (2024). Implementation of explanatory texts output for bridge damage in a bridge inspection web system. *Advances in Engineering Software*, 195, 103706.



- Chun, P.-J., Yamane, T., & Maemura, Y. (2022). A deep learning-based image captioning method to automatically generate comprehensive explanations of bridge damage. *Computer-Aided Civil and Infrastructure Engineering*, 37(11), 1387–1401.
- Chung, S.-W., Hong, S.-S., & Kim, B.-K. (2023). Performance comparison of deep learning models for damage identification of aging bridges. *Applied Sciences*, 13(24), 13204. <https://doi.org/10.3390/app132413204>
- Flotzinger, J., Rösch, P. J., & Braml, T. (2024). dacl10k: benchmark for semantic bridge damage segmentation. In *Proceedings of the IEEE/CVF winter conference on applications of computer vision* (pp. 8626–8635). IEEE Computer Society.
- Giglioni, V., Venanzi, I., Poggioni, V., Milani, A., & Ubertini, F. (2023). Autoencoders for unsupervised real-time bridge health assessment. *Computer-Aided Civil and Infrastructure Engineering*, 38(8), 959–974.
- Graves, A., Fernández, S., & Schmidhuber, J. (2005). Bidirectional LSTM networks for improved phoneme classification and recognition. In Cabestany, J., Prieto, A., & Sandoval, F. (Eds.), *Proceedings of the 8th international conference on Artificial Neural Networks: computational Intelligence and Bioinspired Systems*, pp. 799–804. Berlin, Heidelberg: Springer Berlin Heidelberg.
- Graybeal, B. A., Phares, B. M., Rolander, D. D., Moore, M., & Washer, G. (2002). Visual inspection of highway bridges. *Journal of nondestructive evaluation*, 21(3), 67–83.
- Hoskere, V. (2020). *Multi-task semantic segmentation of damage and materials for visual inspection of civil infrastructure* (M.S. thesis, University of Illinois at Urbana-Champaign). <https://hdl.handle.net/2142/108191>
- Hoskere, V., Hassanlou, D., Rahman, A. U., Bazrgary, R., & Ali, M. T. (2025). Unified framework for digital twins of bridges. *Automation in Construction*, 175, 106214. <https://doi.org/10.1016/j.autcon.2025.106214>
- Huang, Y.-H. (2010). Artificial neural network model of bridge deterioration. *Journal of Performance of Constructed Facilities*, 24(6), 597–602. [https://doi.org/10.1061/\(ASCE\)CF.1943-5509.0000124](https://doi.org/10.1061/(ASCE)CF.1943-5509.0000124)
- Hüthwohl, P., Brilakis, I., Borrmann, A., & Sacks, R. (2018). Integrating rc bridge defect information into BIM models. *Journal of Computing in Civil Engineering*, 32(3), 04018013.
- Iakubovskii, P. (2019). *Segmentation models pytorch*. https://github.com/qubvel/segmentation_models.pytorch. GitHub.
- Javadinasab Hormozabad, S., Gutierrez Soto, M., & Adeli, H. (2021). Integrating structural control, health monitoring, and energy harvesting for smart cities. *Expert Systems*, 38(8), e12845.
- Karaaslan, E., Bağcı, U., & Catbas, F. N. (2019). Artificial intelligence assisted infrastructure assessment using mixed reality systems. *Transportation Research Record*, 2673(12), 413–424.
- Kim, B., & Cho, S. (2020). Automated multiple concrete damage detection using instance segmentation deep learning model. *Applied Sciences*, 10(22), 8008.
- Kirchhoff, Y., Rokuss, M. R., Roy, S., Kovacs, B., Ulrich, C., Wald, T., Zenk, M., Vollmuth, P., Kleesiek, J., Isensee, F., & Maier-Hein, K. (2024). Skeleton recall loss for connectivity conserving and resource efficient segmentation of thin tubular structures. In Leonardis, A., Ricci, E., Roth, S., Russakovsky, O., Sattler, T., Varol, G. (Eds.), *Computer Vision – ECCV 2024*. ECCV 2024. Lecture Notes in Computer Science, vol 15135, pp. 218–234. Springer, Cham. https://doi.org/10.1007/978-3-031-72980-5_13
- Kirillov, A., Mintun, E., Ravi, N., Mao, H., Rolland, C., Gustafson, L., Xiao, T., Whitehead, S., Berg, A. C., Lo, W.-Y., Dollár, P., & Girshick, R. (2023). Segment anything. In *Proceedings of the IEEE/CVF international conference on computer vision* (pp. 4015–4026). IEEE.
- Kong, X., Li, Z., Zhang, Y., & Das, S. (2022). Bridge deck deterioration: Reasons and patterns. *Transportation Research Record: Journal of the Transportation Research Board*, 2676(7), 570–584. <https://doi.org/10.1177/03611981221080140>
- Kruachottikul, P., Coocharojananone, N., Phanomchoeng, G., & Kovitanggoon, K. (2021). Development of a user-centric bridge visual defect quality control assisted mobile application: A case of Thailand's department of highways. *Applied Sciences*, 11(20), 9555. <https://doi.org/10.3390/app11209555>
- Kunlamai, T., Yamane, T., Suganuma, M., Chun, P.-J., & Okatani, T. (2024). Improving visual question answering for bridge inspection by pre-training with external data of image–text pairs. *Computer-Aided Civil and Infrastructure Engineering*, 39(3), 345–361.
- La, H. M., Gucunski, N., Kee, S.-H., & Nguyen, L. V. (2015). Data analysis and visualization for the bridge deck inspection and evaluation robotic system. *Visualization in Engineering*, 3(1), Article 6. <https://doi.org/10.1186/s40327-015-0017-3>
- Landi, F., Marsili, F., Keßler, S., & Croce, P. (2025). An unsupervised learning approach to predict the deterioration of aging bridges using inspection data. *Structural Concrete*, 26(5), 5275–6822.
- Li, S., Zhang, Z., Lin, D., Zhang, T., & Han, L. (2023). Development of a BIM-based bridge maintenance system (BMS) for managing defect data. *Scientific Reports*, 13(1), 846.
- Li, T., Alipour, M., & Harris, D. K. (2021). Context-aware condition information extraction from bridge inspection reports using bi-directional LSTM-CRF sequence labeling. *Advanced Engineering Informatics*, 49, 101333. <https://doi.org/10.1016/j.aei.2021.101333>
- Liu, J., Li, D., & Xu, X. (2024). Enhancing bridge damage detection with mamba-enhanced hrnet for semantic segmentation. *PLOS One*, 19(10), e0312136. <https://doi.org/10.1371/journal.pone.0312136>
- Liu, J., Niu, P., Kou, L., Zhang, Y., Chang, H., & Guo, F. (2025). An interactive cross-multi-feature fusion approach for salient object detection in crack segmentation. *Computer-Aided Civil and Infrastructure Engineering*, 40(8), 1080–1099.
- Liu, K., & El-Gohary, N. (2017). Ontology-based semi-supervised conditional random fields for automated information extraction from bridge inspection reports. *Automation in Construction*, 81, 313–327. <https://doi.org/10.1016/j.autcon.2017.02.003>
- Lu, R., & Brilakis, I. (2019). Digital twinning of existing reinforced concrete bridges from labelled point clusters. In Al-Hussein, M. (Ed.), *Proceedings of the 36th International Symposium on Automation and Robotics in Construction (ISARC 2019)*, pp. 616–623, Curran Associates, Inc. <https://doi.org/10.22260/ISARC2019/0082>
- Luleci, F., Catbas, F. N., & Avci, O. (2022). A literature review: Generative adversarial networks for civil structural health monitoring. *Frontiers in Built Environment*, 8, 1027379.
- Marsili, F., Landi, F., Hajdin, R., & Kessler, S. (2025). Bias-aware degradation models for reinforced concrete bridges based on XAI. *Developments in the Built Environment*, 21, 100617.
- Mundt, M., Majumder, S., Murali, S., Panetsos, P., & Ramesh, V. (2019). Meta-learning convolutional neural architectures for multi-target concrete defect classification with the concrete defect



- bridge image dataset. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 11196–11205.
- Perez-Ramirez, C. A., Amezquita-Sanchez, J. P., Valtierra-Rodriguez, M., Adeli, H., Dominguez-Gonzalez, A., & Romero-Troncoso, R. J. (2019). Recurrent neural network model with Bayesian training and mutual information for response prediction of large buildings. *Engineering Structures*, 178, 603–615.
- Qin, S., Qi, T., Deng, T., & Huang, X. (2024). Image segmentation using vision transformer for tunnel defect assessment. *Computer-Aided Civil and Infrastructure Engineering*, 39(21), 3243–3268.
- Rafiei, M. H., & Adeli, H. (2017). A novel machine learning-based algorithm to detect damage in high-rise building structures. *Structural Design of Tall and Special Buildings*, 26(18), e1400.
- Rafiei, M. H., & Adeli, H. (2018). A novel unsupervised deep learning model for global and local health condition assessment of structures. *Engineering Structures*, 156, 598–607.
- Rahman, A. U., & Hoskere, V. (2025). Instance segmentation of reinforced concrete bridge point clouds with transformers trained exclusively on synthetic data. *Automation in Construction*, 173, 106067. <https://doi.org/10.1016/j.autcon.2025.106067>
- Rakoczy, A. M., Ribeiro, D., Hoskere, V., Narazaki, Y., Olaszek, P., Karwowski, W., Cabral, R., Guo, Y., Futai, M. M., Milillo, P., Santos, R., Trias, A., Gonzalez, L., Matos, J. C., & Schmidt, F. (2024). Technologies and platforms for remote and autonomous bridge inspection—Review. *Structural Engineering International*, 35(3), 1–23. <https://doi.org/10.1080/10168664.2024.2368220>
- Ranjith, S., Setunge, S., Gravina, R. J., & Venkatesan, S. (2013). Deterioration prediction of timber bridge elements using the Markov chain. *Journal of Performance of Constructed Facilities*, 27(3), 319–325. [https://doi.org/10.1061/\(ASCE\)CF.1943-5509.0000311](https://doi.org/10.1061/(ASCE)CF.1943-5509.0000311)
- Rashidi Nasab, A., & Elzarka, H. (2023). Optimizing machine learning algorithms for improving prediction of bridge deck deterioration: A case study of Ohio bridges. *Buildings*, 13(6), 1517. <https://doi.org/10.3390/buildings13061517>
- Ravi, N., Gabeur, V., Hu, Y.-T., Hu, R., Ryali, C., Ma, T., Khedr, H., Rädle, R., Rolland, C., Gustafson, L., Mintun, E., Pan, J., Alwala, K. V., Carion, N., Wu, C.-Y., Girshick, R., Dollár, & P. Feichtenhofer, C. (2024). Sam 2: Segment anything in images and videos. In *Proceedings of the Thirteenth International Conference on Learning Representations (ICLR 2025)*.
- Ronneberger, O., Fischer, P., & Brox, T. (2015). U-net: Convolutional networks for biomedical image segmentation. In *Medical image computing and computer-assisted intervention—MICCAI 2015: 18th international conference, proceedings, part III 18* (pp. 234–241). Munich, Germany, October 5–9, 2015.
- Rosso, M. M., Aloisio, A., Randazzo, V., Tanzi, L., Cirrincione, G., & Marano, G. C. (2024). Comparative deep learning studies for indirect tunnel monitoring with and without Fourier pre-processing. *Integrated Computer-Aided Engineering*, 31(2), 213–232.
- Sacks, R., Kedar, A., Borrmann, A., Ma, L., Brilakis, I., Hühthwohl, P., Daum, S., Kattel, U., Yosef, R., Liebich, T., Barutcu, B. E., & Muhic, S. (2018). Seebridge as next generation bridge inspection: Overview, information delivery manual and model view definition. *Automation in Construction*, 90, 134–145.
- Sajedi, S., & Liang, X. (2022). Deep generative Bayesian optimization for sensor placement in structural health monitoring. *Computer-Aided Civil and Infrastructure Engineering*, 37(9), 1109–1127.
- Spencer Jr, B. F., Hoskere, V., & Narazaki, Y. (2019). Advances in computer vision-based civil infrastructure inspection and monitoring. *Engineering*, 5(2), 199–222. <https://doi.org/10.1016/j.eng.2018.11.030>
- Wang, G., Huang, H., Wang, J., Wang, Y., Yi, Y., Gong, G., & Zhou, G. (2025). A central difference attention multi-modal segmentation network integrating continual learning and graph convolution diffusion algorithms for complex road crack segmentation. *Computer-Aided Civil and Infrastructure Engineering*, 1–25. <https://doi.org/10.1111/mice.70076>
- Xia, T., Yang, J., & Chen, L. (2022). Automated semantic segmentation of bridge point cloud based on local descriptor and machine learning. *Automation in Construction*, 133, 103992. <https://doi.org/10.1016/j.autcon.2021.103992>
- Ye, C., Butler, L. J., Calka, B., Iangurazov, M., Lu, Q., Gregory, A., Girolami, M., & Middleton, C. (2019). A digital twin of bridges for structural health monitoring. In Chang, F.-K. (Ed.), *Enabling Intelligent Life-Cycle Health Management for Industry Internet of Things (IIOT)*, *Proceedings of the Twelfth International Workshop on Structural Health Monitoring*, pp. 1619–1626. DEStech Publications, Inc. <https://doi.org/10.12783/shm2019/32287>
- Yu, Z., Dai, C., Zeng, X., Lv, Y., & Li, H. (2025). A lightweight semantic segmentation method for concrete bridge surface diseases based on improved deeplabv3+. *Scientific Reports*, 15(1), 10348. <https://doi.org/10.1038/s41598-025-95518-5>
- Zaker Esteghamati, M., Hassan Ahmed, M., Young, S., & Halling, M. (2024). A multi-task deep learning-based framework for structural details and corrosion detection in steel bridges. In Jensen, J. S., Frangopol, D. M., Schmidt, J. W. (Eds.), *Bridge Maintenance, Safety, Management, Digitalization and Sustainability*, pp. 3530–3537. CRC Press. <https://doi.org/10.1201/9781003483755-417>
- Zhou, Z., Zhang, J., & Gong, C. (2023). Hybrid semantic segmentation for tunnel lining cracks based on swin transformer and convolutional neural network. *Computer-Aided Civil and Infrastructure Engineering*, 38(17), 2491–2510.
- Zhu, G., Liu, J., Fan, Z., Yuan, D., Ma, P., Wang, M., Sheng, W., & Wang, K. C. (2024). A lightweight encoder–decoder network for automatic pavement crack detection. *Computer-Aided Civil and Infrastructure Engineering*, 39(12), 1743–1765.

How to cite this article: Çelik, R. F., Hoskere, V., & Kessler, S. (2025). From pixels to rating—A semiautomated system linking multi-damage segmentation and condition rating in concrete bridge inspections. *Computer-Aided Civil and Infrastructure Engineering*, 1–25. <https://doi.org/10.1111/mice.70145>

APPENDIX: BMS DATA PREPROCESSING

The CSV data from the BMS underwent multiple pre-processing steps to prepare it for ML training. Initially, irrelevant columns and damage types other than the previously selected 22 were filtered out as described in Subsection 5.1. In the next step, the data were cleaned and simplified by removing excessive whitespace, tabs, quotes,

TABLE A.1 The table shows how the damage type terms in the column *damage type* are processed to align the previously defined damage class labels. Some terms from the column *damage type* are prepend to the input in *(free-)text* which is represented by [text].

damage type	(free-)text		damage type	(free-)text
spalled	[text]	→	spalling	[text]
moisture ingressed	[text]		moisture ingress	[text]
moisture ingressed with efflorescence	[text]		moisture ingress	with efflorescence, [text]
with efflorescence	[text]		crack	with efflorescence, ...
with moisture ingress and efflorescence	[text]		crack	with moisture ingress and efflorescence, [text]
with discoloration of the crack banks	[text]		crack	with rust flag, [text]
with moisture ingress and discoloration	[text]		crack	with moisture ingress and rust flag, [text]
with moisture ingress	[text]		crack	with moisture ingress, [text]
0.1 mm ≤ 0.2 mm	[text]		crack	with 0.1 mm ≤ 0.2 mm, [text]
0.2 mm ≤ 0.4 mm	[text]		crack	with 0.2 mm ≤ 0.4 mm, [text]
0.4 mm ≤ 1 mm	[text]		crack	with 0.4 mm ≤ 1 mm, [text]
≥ 1 mm	[text]		crack	with ≥ 1 mm, [text]
torn	[text]		crack	[text]
cracked	[text]		crack	[text]

and redundant commas and by normalizing special characters (e.g., the German character β was changed to its alternative form *ss*). We removed all samples with *component extension* input other than the following entries: *Covering, spacer, general, anchor, face surface, concrete, concrete B II, concrete cover, concrete replacement system, surface coating, diagonal braces, top view, epoxy resin, epoxy mortar, tile, structure, structural texture, curb stone, head beam, artificial stone, mortar, natural stone, surface protection system for concrete, surface, surface texture, paving stone, pavement, slab, plaster, plaster on concrete, reactive resin mortar/concrete, vertical surface, other surface protection system, shotcrete B II, spray mortar/concrete with polymer additive, stone, stone facing, load-bearing masonry, stair, facing brickwork, and cement mortar/concrete with polymer additive*.

Then, we modified the damage type terms in the column *damage type* to unify them and facilitate NLP model training. Specifically, we did the following: (1) For all damage types that are a crack type, we replaced the terms in *damage type* by the damage class label *crack* and prepend crack specifications, like crack width, to the column *(free-)text*. (2) We also modified the names of the *damage type* terms. If the word stem is the same as the word stem of the equivalent damage class label but the affix is different, we adapted the damage type term to the equivalent damage class label, for example, *spalled* became *spalling*. (3) We also exchanged the damage type terms *discoloration* and *discoloration of the crack banks* by the damage class label *rust flag* since these damage types align visually. The changes are outlined in Table A.1 where the original German terms are translated to English.

In the next preprocessing step, the whole data were checked for redundancies and abbreviations, especially in the *(free-)text* column, which has many inconsistent abbreviations. To make the text consistent, we analyzed the text and collected all abbreviations that we identified. Then we analyzed the collected abbreviations and defined consistent terms to represent a group of abbreviations that mean the same. These consistent terms were either an abbreviation or the full form of the abbreviation group. A case in point is the German word *freiliegende*, which translates to *exposed* as in *exposed rebar* (in German *freiliegende Bewehrung*). In the data, we found five abbreviations for *freiliegende* which are *frei.*, *freil.*, *freil*, *Frei lieg.*, and *fr. lieg.* which were all expanded to *freiliegende*. Another example concerns the word *südlich* which translates to *southern* in English. German language has a rich inflectional morphology where adjectives are based on case, gender, and number to align them with the noun they describe. So, the adjective *südlich* appears in further inflectional forms like *südliche*, *südlichen*, and *südliches*. However, it appears also in abbreviated form as *südl*. So, we replaced all forms of the adjective by the abbreviation *südl*. to make the tokenization process later less complex. Some abbreviations were not written out since many times it was not possible to write them out due to ambiguity. For example, the words *Pfosten* (in English *post*) and *Pfeiler* (in English *pillar*) are both abbreviated by *Pf*. These abbreviations were kept as they are. For reasons of brevity and clarity, we provide only few but relatively complex representative examples. Many abbreviations were easier to handle than the ones presented here, though the challenge was to find out the full form of abbreviations that represent technical terms.



In the next step, we checked the terms used in the column *general damage quantity*. We extracted 22 general quantity terms. These are standard fixed terms in the German BMS which bridge inspectors have to choose from. The German guidelines for bridge management differentiate actually three quantity types: *small*, *medium*, and *large*. So, the 22 terms were allocated to one of the main quantity types. The quantity type *small* includes terms such as “isolated” (German: *vereinzelt*), “single location” (*eine Stelle*), “minor” (*gering*), “localized” (*punktuell*), “a piece” (*ein Stück*), and “starting to form” (*beginnend*). The quantity type *medium* includes terms such as “multiple” (*mehrfach*), “spotwise (stellenweise)”, “in certain areas” (*bereichsweise*), “partially” (*teilweise*), “pronounced” (*ausgeprägt*), “numerous” (*zahlreich*), and “on several structural elements” (*an einigen Bauteilen*). The quantity type *large* includes terms such as “extensive” (*großflächig*), “frequent” (*häufig*), “continuous” (*durchgehend*), “largely” (*größtenteils*), “entire structural element” (*gesamtes Bauteil*), “on all structural elements” (*an allen Bauteilen*), “fully” (*vollständig*), “all” (*alle*), and “widespread” (*flächendeckend*). After this allocation, we replaced these 22 terms by the corresponding quantity type terms, which in turn were converted to numbers 0 (small), 1 (medium), and 2 (large). This way, we created a more streamlined data.

Next, the data were lowercased. Then, two new columns were created to separate input and output data for the supervised NLP training: *input_text* and *output*. The

input_text column contains a space separated string concatenation of the inputs from the columns *component*, *structural element*, *component extension*, *component group*, *general damage quantity*, *(free-)text*, *damage type*, and *damage progress*. The *output_text* column contains a comma separated string concatenation of the condition rating values from the columns *structural stability*, *traffic safety*, and *durability*.

After streamlining the data, we discovered that some *input_text* samples were identical, while their corresponding *output* labels differed by typically a single score in one of the rating criteria. This discrepancy can have multiple causes, among which three are particularly relevant to consider. First, the textual description cannot fully capture the severity of the damage, as minor differences in rating can result from visual cues not being reflected in the text. Second, the data were generated by different inspectors, each applying their own judgment and experience, leading to subjective variation. Third, in certain cases, inspectors may deliberately assign a more severe rating to emphasize the urgency of maintenance measures to bridge owners.

However, such ambiguity in the data can lead to unstable model training and reduce the reliability of predictions. To address this, we performed a unification step: For all identical *input_text* samples with differing *output* labels, we retained only the version with the worst rating since it is preferable to overestimate damage severity rather than underestimate it.